UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

MASTER THESIS

Magnetic Systems for the ADCS of a Femtosatellite

Manuel Cachón Vigil

SUPERVISED BY

Jordi Gutiérrez Cabello

Universitat Politècnica de Catalunya
Master in Aerospace Science & Technology
February 2023

Magnetic systems for the ADCS of a femtosatellite

BY

Manuel Cachón Vigil

DIPLOMA THESIS FOR DEGREE

Master in Aerospace Science and Technology

AT

Universitat Politècnica de Catalunya

SUPERVISED BY:

Jordi Gutiérrez Cabello

Department of Physics

ABSTRACT

This project aims to select the most suitable ADCS for a femtosatellite intended to measure the atmosphere at low orbital altitudes (under 500 km); to do so, we will employ a MATLAB code that models the systems performance in terms of total rotation speed, which is targeted to be less than a degree per second after a week. To that end, two systems are studied and compared. Those are hysteresis material and a set of magnetorquers.

In order to model their operation, the orbit is propagated using two models (SGP4 and SGP8). The starting position for them is given by means of a TLE, and it is the same for all of them. Then, with the propagated new locations of the satellite, the local magnetic field for each one of them is calculated by means of two widely-used numerical models (WMM and IGRF). This generates four data sets for the magnetic field obtained with four slightly different modelling techniques that provide a way to get reliable results.

These four data sets are coupled with a simulation of the satellite's attitude that starts with a rather large initial rotation speed (20 degrees per second in each axis of the body reference frame; these conditions are the standard worst case for satellite ejection from the launcher). That is modified by the actuation of one of the proposed attitude systems on the satellite as a function of the local magnetic field.

A permanent magnet would never stabilise the rotation, and so it is dismissed as not useful. The hysteresis material cancels the rotation in less than5 hours, reducing the time that takes to achieve stability the lower the orbit is. Regarding the magnetorquers, they achieve stability in under 1 hour, and in most of the cases in under 30 minutes.

The data obtained clearly shows that the fastest detumbling method are the magnetorquers as the study of a COTS not optimised model achieves a stabilisation time is an order of magnitude faster than the next best method, is compliant with the requirement of being less than 200 g and having a peak power consumption of less than 250mW, and after detumbling less than 10 mW.

To my family and Nadia.

Table of Contents

List of abbreviations

ADCS	Attitude Determination and Control System
BRF	Body Reference Frame
ECEF	Earth Centred Earth Fixed
ECI	Earth Centred Inertial
FAI	Fédération Aéronautique Internationale
GPS	Global Positioning System
IAGA	International Association of Geomagnetism and Aeronomy
IGRF	International Geomagnetic Reference Field
IMU	Inertial Measurement Unit
IS	International System
ISA	International Standard Atmosphere
LEO	Low Earth Orbit
LLA	Longitude Latitude Altitude
ME	Mechanical Energy
NATO	North Atlantic Treaty Organization
NED	North East Dow
NORAD	North American Aerospace Defense Command
SGP	Simplified General Perturbations
TLE	Two-Line Element
UK	United Kingdom of Great Britain and Northern Ireland
UN	United Nations
USA	United States of America
USAF	United States Air Force
USSF	United States Space Force
UTC	Universal Time Coordinated
WMM	World Magnetic Model

List of Figures

List of tables

Chapter 1

INTRODUCTION

1.1. Atmosphere

The Spanish Language Royal Academy defines the atmosphere as “the gaseous layer that surrounds the Earth and other celestial bodies”. Its composition and structure are different in each case, as the distance to the parent star, and size of the body –among other factors– influence its evolution and behaviour. For the Earth, the atmosphere is composed mainly of nitrogen (78%) and oxygen (21%) with trace amounts of other gases like argon or carbon dioxide (CO_2).

Nevertheless, in every celestial body the atmosphere experiences a decrease in pressure, and therefore density, as a function of the height from the surface. This decrease of pressure in an isothermal atmosphere has a logarithmic relation with the altitude increase, following**1.1**

$$\Delta p = e^{\frac{-Mg\Delta h}{RT}} \qquad 1.1$$

where M is the molecular mass, g is the acceleration of gravity,Δh is the variation in altitude, R is the ideal gas constant and T is the average temperature between both heights. This formula can give us a gross approximation of the values, but real atmospheres are not isothermal, and in fact this variation of temperature with height allows us to stratify it into several layers as shown in **Table 1.1** for the case of the Earth [1].

A static standard model called International Standard Atmosphere (ISA) of how pressure, temperature, density and viscosity changes with altitude was developed in the 1970s. It includes tables of values and the formulas that approximate the atmospheric properties, and it is vital for aviation, as it heavily relies on atmospheric conditions. Nevertheless, this model is only reliable for the lower and denser layers of the atmosphere, and predicting its properties in space is far beyond its capabilities.

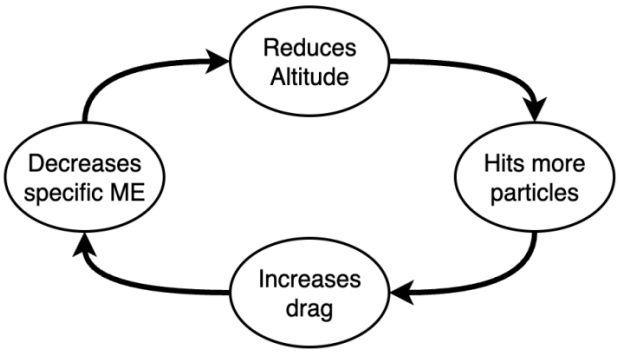

Figure 1.1 Atmospheric drag positive feedback loop. ME stands for Mechanical Energy.

We consider that space begins at the von Kármán line, established at 100 km by the Fédération Aéronautique Internationale (FAI), and most of the regulatory agencies like the UN accept this value or something close to it as the boundary of space. And although most satellites' orbits will always remain well above this boundary, there remains still enough gas in LEO to affect their dynamical behaviour. This means that any spacecraft inside the thermosphere (the layer between 80 km and about 500-600 km above sea level) will experience frequent collisions with particles inducing a positive feedback loop (**Figure 1.1**) that ultimately causes the spacecraft to burn on re-entry.

Table 1.1 Layers of the atmosphere

Layer	Heights	Characteristics
Troposphere	0-12 km	Decrease in temperature with height driven by surface heating. 80% of the atmosphere mass. Most of the atmospheric weather occurs here. Only layer available for propeller driven aircraft.
Stratosphere	12-50 km	Increase in temperature with height driven by UV radiation. Stable conditions, lack of turbulence induce weather. Contains the ozone layer. Highest layer accessible by jet powered aircraft.
Mesosphere	50-80 km	Decrease in temperature with height. Coldest place on Earth –85ºC Most meteors burn up here. Only accessible by sounding rockets and rocket propelled aircraft.
Thermosphere	80-700 km	Increase in temperature with height. Height varies considerably due to solar activity. Completely cloudless. Contains most of the ionosphere (where auroras are produced). Molecules travel 1 km on average between collisions with other molecules.
Exosphere	700-10,000 km	Most spacecraft orbit here. Doesn’t behave like a collisional gas. Atoms so far apart that can travel hundreds of km before colliding with other ones.

Therefore, the drag caused by the remaining atmospheric gases has implications all throughout the mission as, for example, by limiting the operational life or restricting the orbits available to ensure that this decay takes longer than the expected satellite’s operational life. It can also impact the architecture of the spacecraft, demanding some kind of propulsion to mitigate this effect, and hence requiring to devote some of the limited mass budget to propellant.

Furthermore, the atmospheric parameters at those altitudes vary with solar cycles, geomagnetic activity, Earth's position around the Sun, and Moon's position around the Earth, among other factors. So, measuring its properties is very important to accurately design space missions

1.2. How to measure the atmosphere

The easiest way to measure the atmosphere at that altitude is by using a satellite of known characteristics, and observe how the orbit gets modified by the atmospheric effects in comparison with the predicted orbit without this drag force. Therefore, this project is focused on a mission that will try to accomplish that with the following base characteristics:

- Spherical femtosatellite
- No more than 400g
- 100 mm in diameter
- Low and fast decaying orbit (initial altitudes in the range 200 – 500 km)
- Near diagonal inertia tensor.

To measure the atmospheric density, we plan to use a very sensible and precise accelerometer to directly determine the drag force experienced by the satellite. To provide accurate position and time information to the accelerometer's measurements, we will employ a GNSS receiver (GPS, GLONASS, and/or GALILEO). Furthermore, these location and time data will provide a backup method to determine along-orbit average densities if the altitude is too large for the accuracy of the accelerometer, or in case of its malfunction.

Nevertheless, the GNSS antenna has a limitation in rotation rate to ensure that it is able to always receive the signal of at least 4 satellites and find its position. Therefore, the femtosatellite requires some ADCS system in order to reduce the angular speed to a manageable one.

1.3. ADCS

There are several methods that can be used to control the attitude, each one with its pros and cons for our system. A general description of them is shown in**Table 1.2**, as well as a short discussion about their suitability for our project.

The analysis shows that most of the listed actuators types are not suitable for the mission, leaving the magnetic methods as the only viable options to be pursued. Of those, we can differentiate three types: permanent magnets, hysteresis rods and magnetorquers. Their general description as well as their pros and cons are shown in **Table 1.3.¡Error! No se encuentra el origen de la referencia.**

Table 1.2 ADCS Methods.

Method	Description	Analysis
Thrusters	Propulsion system that by being fired imparts a torque in a desired axis, therefore controlling the attitude.	Requires propellant which adds mass to the spacecraft and can potentially disrupt the orbit.
Momentum wheels	Wheels that spin up or down to change the angular momentum, and therefore control the attitude along the rotational axis.	Mass and power budget constraints will make this unfeasible.
Control moment gyros	Similar to the momentum wheels, but allow also change the axis in which they rotate, controlling in this way the angular momentum of the spacecraft.	Similar to momentum wheels but adds even more complexity, they are suitable for bigger spacecraft. The most precise attitude control method in existence.
Solar pressure	Solar light imparts a small force on the spacecraft; by controlling the surface exposed to light the satellite can be stabilised.	Is not a significant force in the LEO environment in which the satellite will be located. On the other hand, the satellite's spherical shape will negate this effect.
Aerodynamic	Atmospheric gas imparts a pressure on the spacecraft; by controlling the cross-section of the satellite, it can be stabilised similarly to an aircraft.	The satellite's spherical shape and homogeneous mass distribution will negate this effect.
Magnetic actuators	The satellite can generate a magnetic field to interact with the Earth's one, therefore generating a torque that can be used for ADCS.	Light weight, low power consumption and ideal for low orbits.

Table 1.3 Types of magnetic ADCS.

Method	Description	Analysis
Permanent magnets	The permanent magnet forces the satellite to align itself with the Earth's magnetic field.	There is little to no damping effect and the oscillation generated is similar to a pendulum.
Hysteresis rods	A hard ferromagnetic material that can dissipate energy while rotating through the Earth's magnetic field.	Reduces oscillations of the satellite but is unable to select the orientation; usually, it is combined with a permanent magnet to set said preferred orientation.
Magnetorquers	A current run through metallic coils generates a magnetic field that interacts with the Earth's, adjusting the position.	Precise and active orientation at the expense of power consumption, mass and complexity added to the satellite.

As seen in **Table 1.3** permanent magnets give no stabilization, instead they force the spacecraft to oscillate around the magnetic field lines, therefore it alone is not suitable for our purposes. With regard to the hysteresis rods and magnetorquers, its main difference is that the first one is a very simple passive system that does not require any power, while the second is active but at the cost of added complexity and power required.

Hysteresis rods require to be as slender as they can to get the most damping possible, but the parameter that mainly controls their effect is the volume of the material used.

The magnetorquer strength is directly related with the number of loops present in the loop and how big those are, but the longer the cable is, the bigger the resistance, and the lower its effect.

1.4. Modelling tools used

In order to perform the simulation of the satellite's attitude, we require a set of tools that we describe in the next sections.

1.4.1. Two-Line Elements (TLE)

Two-Line Element Set is a format for encoding and distributing orbital parameters of Earth orbiting objects developed by NORAD in the 1960s and 70s. Its format of two lines of 69 characters is due to its original purpose: to be used with punched cards. For many years, USAF and later USSF have kept track and assigned TLEs of all detectable orbiting objects around the Earth, and most of them are published online making it a *de facto* standard for this type of information.

1.4.2. Simplified perturbation models

Those are a set of five mathematical models used to propagate the orbital state vectors accounting for the main effects of perturbations caused by the Earth and some phenomena found in the LEO environment. Of those numerical models, the Simplified General Perturbations (SGP) models are the ones used for near Earth orbits. In this study we will use the SGP4 [2] and its revised and improved version SGP8 [3].

The principal advantage of these models is a significant reduction in the computational burden associated with orbit propagation.

1.4.3. MATLAB

Abbreviation of "MATrix LABoratory", is a programming and numerical computing platform widely used by engineers and scientists to analyse data, develop algorithms, and create models[4].Initially released in 1984, it has been continually updated and refined, and its capabilities can be enlarged and improved by using Toolboxes and Simulink, its graphical programming environment.

1.4.3.1. Aerospace and mapping toolboxes

These sets of pre-programmed functions are not included in the basic MATLAB version. Aerospace Toolbox provides standards-based tools and functions for analysing the motion, mission, and environment of aerospace vehicles [5] while Mapping Toolbox™ provides algorithms and functions for transforming geographic data and creating map displays [6].

1.4.3.2. Vector Transformations

In order to perform the transformation between certain coordinate frames, both toolboxes have specific commands already implemented that help us in making those calculations. Those will be:

- "`ned2ecefv`" from the mapping toolbox, that allows us to change vectors from North-East-Down reference frame to Earth Centred Earth Fixed.

- "`ecef2eci`" from the aerospace toolbox, allows us to transform the result from the previous command into Earth-Centred-Inertial frame of reference, the same that SGP models provide.

- "`eci2lla`" also from the aerospace toolbox will change ECI vectors into Latitude-Longitude-Altitude required to use the Magnetic models also included inside the toolbox.

1.4.3.3. International Geomagnetic Reference Field (IGRF)

IGRF is a mathematical model that describes Earth's magnetic field and its secular variation produced by IAGA since 1965 and updated every 5 years [7]. The current last updated version, and the one used for this project, is the 13th (IGRF-13) released in December 2019. It is included in MATLAB's Aerospace toolbox.

1.4.3.4. World Magnetic Model (WMM)

The WMM is also a mathematical model that describes Earth's magnetic field and its secular variation, but in this case produced by USA and UK government agencies and it is the standard for NATO [8]. It is also updated every 5 years and is included in MATLAB's Aerospace toolbox. The last available release is the WMM-2020 published also in December 2019.

1.4.4. Quaternions

Quaternions were firstly described by Irish mathematician William Rowan Hamilton in 1843. They are mathematical objects that extend the real number system similar to the complex numbers (in fact, quaternions are hypercomplex numbers) [9]. They consist of two parts, a scalar and a vector (also called the imaginary part). They are often represented in the forms shown in **1.2**

$$q = \begin{pmatrix} q_0 \\ \vec{q} \end{pmatrix} = q_0 + q_1 i + q_2 j + q_3 k \qquad 1.2$$

where q_0, q_1, q_2 and q_3 are real numbers and i, j and k the imaginary units that satisfy equations **1.3**

$$i^2 = j^2 = k^2 = ijk = -1 \qquad 1.3$$

Quaternions have been used in a variety of fields such as physics, engineering, and computer science. In physics and engineering, quaternions are used to represent rotations in three-dimensional space, and as a result, they are often used in aerospace, robotics and computer graphics. This rotation can be performed by following **1.4.**

$$v' = q\ v\ q^* \qquad 1.4$$

where v is the vector that is to be rotated (written as a quaternion with 0 as scalar part and v as vector part),v' is the rotated vector, q is the quaternion and q^* is the conjugate of said quaternion. The conjugate of the quaternion in expression **1.2** is **1.5**.

$$\boldsymbol{q^* \equiv \begin{pmatrix} q_0 \\ -\vec{q} \end{pmatrix} = q_0 - q_1 i - q_2 j - q_3 k} \qquad \textbf{1.5}$$

Then, the rotation of equation **1.4** can be expressed as show by **1.6.**

$$v' = [0, \vec{v} + 2q_0(\vec{q} \wedge \vec{v}) + 2\vec{q} \wedge (\vec{q} \wedge \vec{v})] \qquad 1.6$$

Additionally, the quaternion used to rotate around a known axis $\vec{p}$ an angle θ is given by **1.7**

$$q = \begin{pmatrix} \cos(\theta/2) \\ p_1\ sin(\theta/2) \\ p_2\ sin(\theta/2) \\ p_3\ sin(\theta/2) \end{pmatrix} \qquad 1.7$$

This last expression is the usual tie between the axis-angle and the quaternion representations of rotations.

Chapter 2

Preliminary work, assumptions and requirements

2.1. Scope of the project

This project attempts to model the performance of the ADCS for a femtosatellite whose goal is to measure the atmospheric density in very low Earth orbit (VLEO, thus with altitudes below 250 km). The attitude control will enable continuous connection with GNSS systems by reducing the femtosatellite's rotation rate and therefore allowing density measurements to have accurate time stamps and geographical location information.

To this end, two types of magnetic actuators will be independently studied: hysteresis rods and magnetorquers. The orbit will be modelled from a synthetic TLE generated according to the requirements expressed in section 2.2. In order to accomplish this task without an enormous computational burden, two SGP propagator models will be used: SGP4 and SGP8. The orbital data provided by those will be used to calculate the local magnetic field using two models, WMM and IGRF. The local magnetic field will interact with the actuators inside the satellite, therefore modifying the rotation rate of the spacecraft.

This work will be performed in MATLAB, and the code will be explained in Chapter 3 by means of a flow diagram and a step by step explanation of the processes involved. We will particularise our analysis for each type of system in Chapter 4. Then a verification process will be undertaken using reduced sections of the code, to assess the difference between the two propagators in section 5.1, the difference between magnetic models in section 5.2, and the magnetic field rotations will be checked in section 5.3. Chapter 6 will be devoted to analyse the resulting rotation speeds with each one of the methods for three different orbits differing on their altitude. In section 6.3, the method that achieves the fastest stabilisation time will be selected to be the final one, provided that all the parameters related to it are compliant with the ones given by the requirements; if this is not the case, the next fastest method will be selected.

Finally, we draw our conclusions and identify possible alleys for future research and improvements of the system.

2.1.1. Out of scope topics

The code that is the main part of this project will not include a graphic interface as a way to input the information required for the simulation; those data will be typed in the section allocated to that purpose inside the code and accordingly labelled.

As said before, the orbit modelling is undertaken by using SGP methods, therefore will not consider celestial bodies other than the Earth.

The magnetic field models used are the ones currently valid when this document is being written (IGRF-13 and WMM-2020); therefore, the simulations will not be useful for dates outside the 2020-2025 period, and later or earlier periods will require minor code modifications.

The permanent magnet ADCS will not be studied, as it does not reduce the rotation rate of the satellite, and usually is used alongside hysteresis rods. The integration of these two methods together inside the simulation is also beyond the scope of the project.

Regarding the power consumption, although an estimation of the power consumed is given further modelling should be performed with a more realistic estimation, considering the parameters of the magnetorquer instead of making an approximation.

2.2. Requirements

Any engineering project needs a set of requirements that "*identify a product operational, functional or design characteristic or constraint, which is unambiguous, testable or measurable, and necessary for product or process acceptability*" [10]. Therefore, a set of requirements will be stated to undergo the design process with guarantees that the final result is suitable to accomplish the objective.

Those requirements stated in **Table 2.1** will have an associated code that will identify them further in the process and a description that will define them. As they have to be prioritized, they will be categorized by labelling them as mandatory (M) or desirable (D), meaning that the second type although being desired, could be waived only if its implementation makes impossible to fulfil a requirement of the first type but otherwise has to be implemented. They also can be divided into functional (F) or non functional (NF), this meaning that describe a capability of the system in the first case, and in the second defines an overall quality or attribute of the system.

Table 2.1 Requirement list and its classification.

Code	Description	Type	Priority
R-1	The mass of the satellite shall be under 400g	NF	M
R-2	Satellite shallbe a sphere of 70 mm of radius	NF	M
R-3	The satellite orbit height above mean sea level shall be 400 km±100 km	NF	M
R-4	The satellite orbit inclination shallbe 95±1 deg	NF	M
R-5	The eccentricity shallbe under 0.001	NF	M
R-6	The ADCS system shall have a mass below 100 grams.	NF	M
R-7	The ADCS shall fit inside the satellite	NF	M

Table 2.1 Requirement list and its classification. (Cont)

Code	Description	Type	Priority
R-8	The ADCS subsystem shall maintain at all times the angular velocity below 2 degrees per second in all axis.	F	M
R-9	The ADCS subsystem shall detumble the satellite in a period no longer than seven days.	F	M
R-10	An active system shall control the attitude of the satellite within a maximum error of 5 degrees.	F	D
R-11	The ADCS system shall have a power consumption below 250 mW.	NF	D
R-12	The determination of the angular velocity shall have a maximum error of 0.2 degrees per second.	F	D
R-13	The ADCS subsystem shall fulfill the mission requirements at all times (including eclipse periods).	F	M

2.3. Assumptions

In order to perform this project several assumptions will be considered.

The first one will be that the cartesian axes will be used as the main axis of the satellite, therefore being an orthonormal set.

Secondly, the initial angular speed will be 20 degrees per second in each one of the axes of the satellite. This is the upper limit rotation rate that deployers can impart to satellites during injection; hence, it is the worst case.

The precision of the SGP propagator models will be assumed to be good enough; the error caused by the propagator for periods under a month is assumed to be acceptable as compared with the benefits in terms of computational time of higher accuracy methods. Furthermore, the goal of this study –the analysis of the performance of magnetic attitude control systems in the presence of a variable magnetic field– can be achieved even if the orbit is not very accurate.

Finally, as the magnetic field actuating on the satellite is much lower than the saturation values for most materials, and due to the lack of data to model its behaviour, it is assumed that the relation between coercivity and external magnetic field is linear.

Chapter 3

Orbit and attitude modelling

3.1. Code general description

The main part of this project is to develop a code able to propagate the orbit and the attitude considering the effects of magnetic actuators used as ADCS. To this end, two orbit propagators will be used, SGP4 and SGP8. Then, for each step the magnetic field will be calculated with either IGRF or WMM, so obtaining as a result the evolution of the rotation rate against time, and at what time the satellite is stabilized, if so. The general scheme of the code can be seen in **Figure 3.1**.

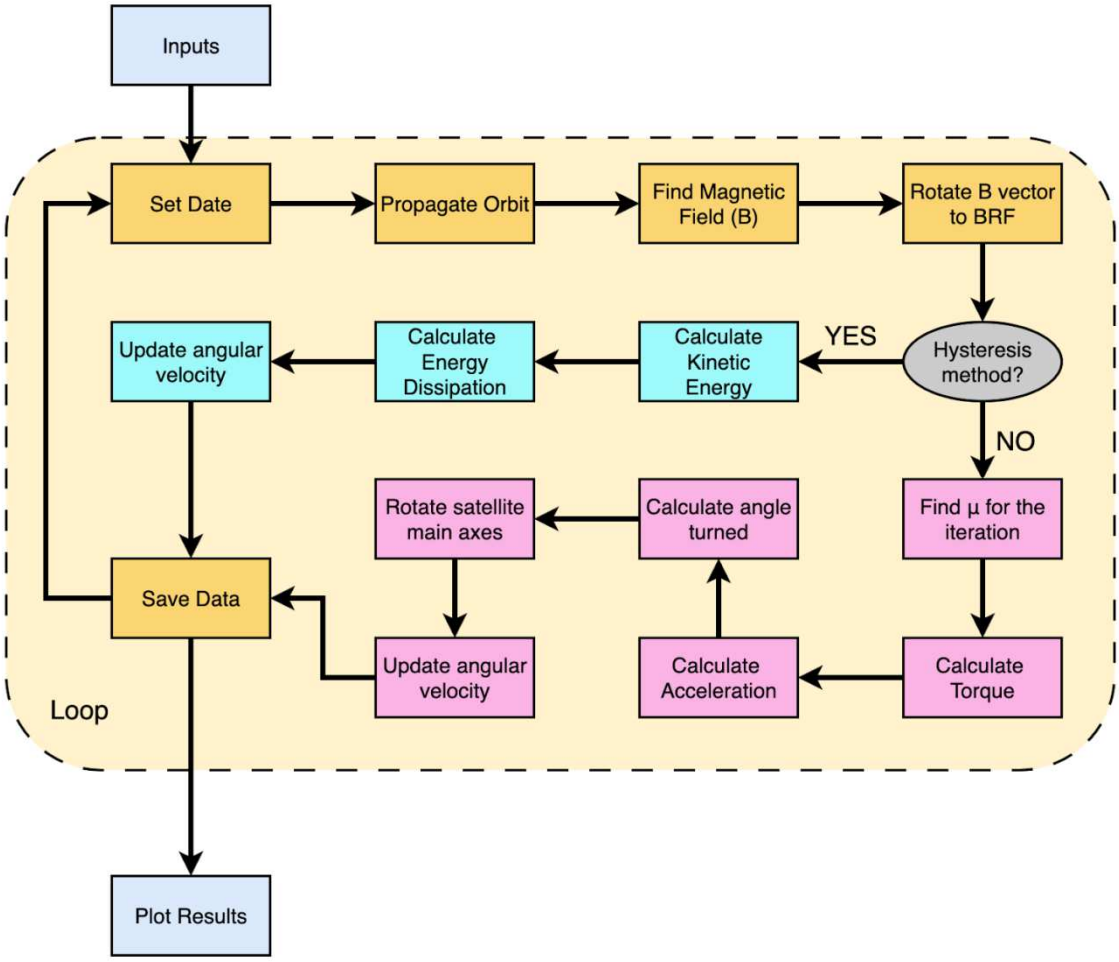

Figure 3.1 Flow diagram of the code.

3.2. Inputs

Here the basic information for the code to run is given by the user:

- Type of actuator: the type of actuator used for the ADCS system is selected by typing "1" if it is hysteresis material, "2" if it is magnetorquers.
- Initial TLE: provided to serve as initial position and time of the satellite from where its orbit will be propagated.
- Inertia tensor: of the satellite or an approximation of the latest design in g mm^2.
- Initial rotation speed: initial rotation speed of the three axes of the satellite given as a vector in ECI frame with units rad/s.
- Propagation parameters: how long the simulated period will be (in days) and how large will be the step taken between iterations (in seconds).
- ADCS characteristics: the different parameters of the ADCS; it will be dependent on the case studied, and the discussion will be followed in Chapter 4.
- Initial attitude: direction for the three main satellite axes (Body Reference Frame, BRF) in ECI given as unit vectors.

3.3. Initial parameters

Generates the storage arrays, defines needed values, initialises counters and makes base calculations and transformations for subsequent processes.

- Reference vector for the ECI frame: generates a unit vector that will be used as a reference for rotations from ECI to other frames.
- Obtain data form TLE: executes the `Orbitalp.m` code in order to get the TLE data as a structured array.
- Adapt inertia tensor units: converts inertia tensor from g mm^2 to kg m^2.
- Adapt time parameters: change the step size and duration of the simulation to minutes, define the first step as 0 and calculate the number of iterations.
- Generate storage arrays: arrays for the angular velocities, the time for each iteration, and for the stabilization dates.
- Initialize iteration counter: its value is set to one.

3.4. Common Loop

This part of the code will keep running while the time elapsed is smaller than the required simulation time.

- Set Date: Generates the date information using the data extracted from the TLE and the propagated time at the current iteration, and provides it as decimal year and UTC format.

- Propagate orbit: propagates the orbit using either SGP4 or SGP8, outputting it in meters and in LLA reference frame.

- Find magnetic field: calculates the magnetic field using IGRF or WMM for the propagated position of the satellite.

- Rotate Magnetic field: from the given NED frame to BRF.

    - Firstly, from NED to ECEF using the command `ned2ecefv` from Matlab's mapping toolbox.

    - Then, from ECEF to ECI using the command `ecef2eci` from Matlab's aerospace toolbox.

    - Finally, from ECI to BRF using a quaternion calculated with the relative orientations of the main axes of the satellite and the ECI frame director vectors.

3.5. Hysteresis Method

If the system evaluated is the use of hysteresis rods, the loop will go through the following steps:

- Calculate rotational kinetic energy: the rotational kinetic energy of the system is calculated by following **3.1**:

$$\boldsymbol{E_k = \frac{1}{2}\vec{\omega}^T\, I\, \vec{\omega}} \qquad \textbf{3.1}$$

- Calculate energy dissipation: then energy dissipated by cycle can be approximated with the magnetic coercivity H_c, the volume of the hysteresis materials V_h, the magnetic field required for full magnetization (B_{Sat}) and the local magnetic field (**3.2**):

$$\boldsymbol{\Delta E_k = 4\, V_h H_c \frac{\|\vec{B}\|}{B_{Sat}}} \qquad \textbf{3.2}$$

- Update angular velocity: with that variation of kinetic energy, the new one can be calculated and therefore the new angular velocity can also be calculated applying **3.1** backwards as shown in **3.3**, where $\vec{S}$ is the axis of rotation (a unit vector) andI_S the moment of inertia calculated around that axis.

$$\vec{\omega} = \vec{S}\sqrt{\frac{2(E_k - \Delta E_k)}{I_S}} \qquad \textbf{3.3}$$

- Verify stabilization: check if angular velocity is below the maximum acceptable value given in the requirement, and save the time at which this happens. If all four models have done so, stop the loop from iterating any further.

3.6. Magnetorquers

In the case in which the ADCS is based on a magnetorquer, the loop will perform the following steps:

- Find μ: as the magnetic field in BRF changes from one iteration to the next, to find this value firstly we will calculate the target acceleration in order to stop the rotation in a given Δt (**3.4**):

$$\vec{\alpha} = \frac{-\vec{\omega}}{\Delta t} \qquad \textbf{3.4}$$

  Then we will calculate the torque required with **3.5**.

$$\vec{T} = I\,\vec{\alpha} \qquad \textbf{3.5}$$

  And then the dipolar moment is given by**3.6**.

$$\vec{\mu} = \|\vec{\mu}\|\hat{\mu} \qquad \textbf{3.6}$$

  where $\hat{\mu}$ is the unit vector in the direction in which $\vec{\mu}$ is pointed at, and $\|\vec{\mu}\|$ is given by equation **3.7**.

$$\|\vec{\mu}\| = \frac{\|\vec{T}\times\vec{B}\|}{\|\vec{K}\|B^2} \qquad \textbf{3.7}$$

  $\vec{K}$ is given by**3.8**

$$\vec{K} = \frac{(\hat{\mu}\cdot\vec{B})\vec{B}}{B^2} - \hat{\mu} \qquad \textbf{3.8}$$

- Calculate the Magnetic Torque: by doing the cross product between the dipolar moment of the satellite and the local magnetic field (**3.9**)

$$\vec{T}_i = \vec{\mu}_i \times \vec{B}_i \quad \textbf{3.9}$$

- Calculate angular acceleration: The angular acceleration can be found solving the linear system generated by the inertia tensor and the torque (**3.10**).

$$\vec{T}_i = I\vec{\alpha}_i \quad \textbf{3.10}$$

- Calculate angle turned: using the equation of the uniformly accelerated circular motion, the angular acceleration and the angular rotation at the beginning of the iteration (**3.11**).

$$\vec{\theta}_{i+1} = \vec{\omega}_i \Delta t + \frac{1}{2}\vec{\alpha}_i \Delta t^2 \quad \textbf{3.11}$$

- Rotate satellite main axis: using a quaternion to represent the rotation depicted by the previous step, being the axis the rotation vector in unit form.

- Update angular velocity: using the uniformly accelerated circular motion, the previous angular rotation, the angular acceleration and the time increment (**3.12**).

$$\vec{\omega}_{i+1} = \vec{\omega}_i + \vec{\alpha}_i \Delta t \quad \textbf{3.12}$$

3.7. End of the loop

- Save data: store the rotation speed and time data into arrays that can be used afterwards outside the loop.

- Update the counters: add one to the constants that iterate the different elements inside the loop to start the new iteration. It also prints on the screen the current percentage of the total iterations scheduled already performed.

3.8. Plot

The data stored in the arrays is plotted to show the performance of the method; this includes angular speeds in the three axes, as well as the module of the angular speed vector for each one of the 4 scenarios. It also prints the time when stabilization has occurred in the hysteresis method.

Chapter 4

Potential Attitude Systems

4.1. Hysteresis Rods

4.1.1. Description of the system

When a magnetic field from an external source is applied to a ferromagnetic material, its atomic structure (the atomic magnetic dipoles) aligns with it, part of it remaining in that aligned state after the field its removed. To completely demagnetize the material, heat (causing atomic vibrations that destroy the alignment) or a magnetic field in the opposite direction is required.

Because the relationship between field strength *H* and magnetization *m* is not linear in such materials (**Figure 4.1**) and causes some internal friction, it can be used to slow the rotation of the satellite following equation **3.2**.

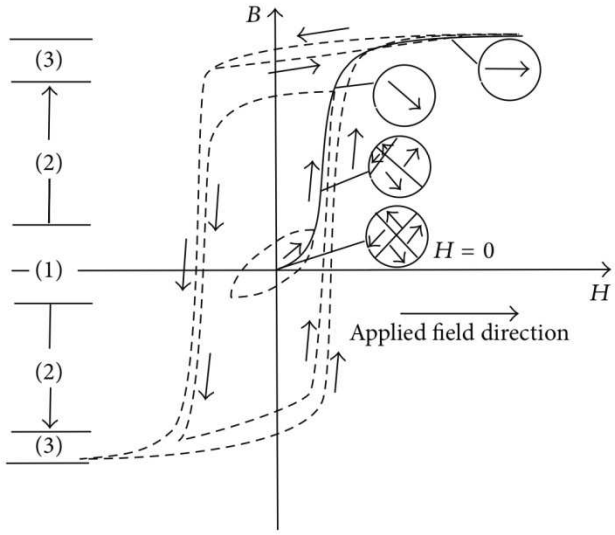

Figure 4.1 Theoretical model of magnetization against magnetic field. Figure taken from [11].

In our case, the hysteresis material is taken to be AlNiCo with a magnetic coercivity H_c of 17.27 A/m and a density of 8.25 g/cm^3; we will assume that it is distributed in 2 rods of 5 cm in length, providing a total volume of 5 cm^3, therefore amounting to a total mass of 41.25g.

It should be noted that the saturation magnetic induction for this material is 1.53 T, much larger than the local terrestrial magnetic field at the altitudes in which the satellite operates. Due to the lack of reliable data to model the real partial magnetisation curve, it is assumed to be linear

$$\frac{\vec{H}_c}{\vec{B}_s} = \frac{\vec{H}_R}{\vec{B}_E} \qquad \textbf{4.1}$$

where $\vec{H}_c$ and $\vec{H}_R$ are the coercivity in the cases of saturating magnetic induction ($\vec{B}_s$) and terrestrial magnetic induction ($\vec{B}_E$).

4.1.2. Simulation details

In this case the rotational speed is updated by calculating the kinetic energy of rotation of the satellite (**3.1**), then the energy dissipated in one cycle (**3.2**), then it is checked if its new kinetic energy is positive (that is, whether $E_k - \Delta E_k > 0$) and the new rotation rate is calculated with equation **3.3**. If the rotational kinetic energy is negative (a physically unrealistic situation), the satellite is considered to be stabilized, the date is stored, and once the four models are stable the simulation is stopped.

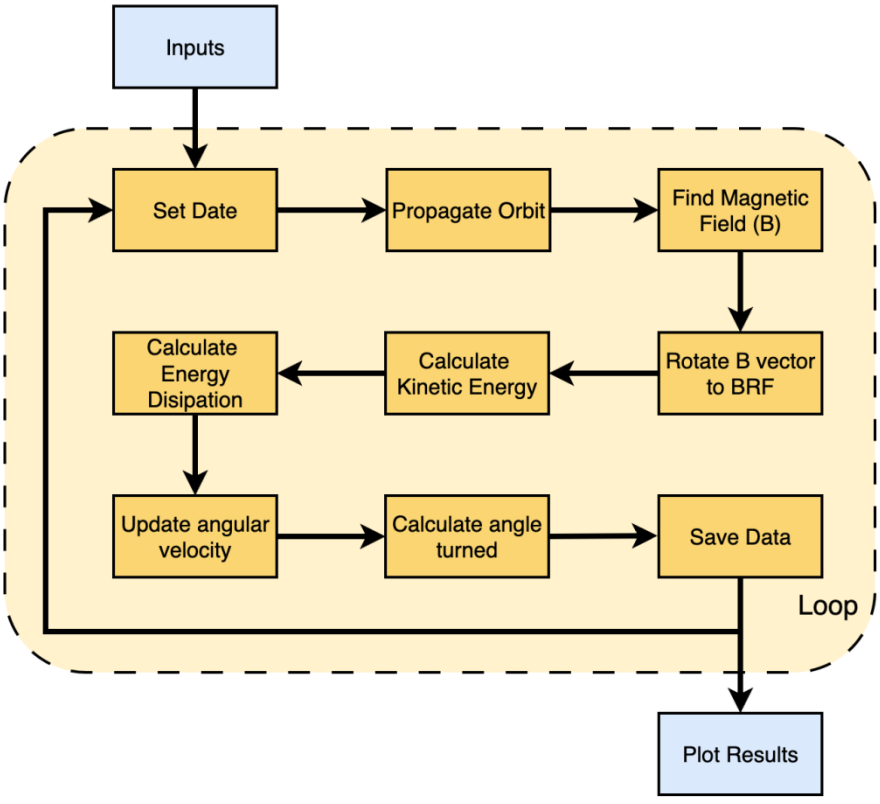

Figure 4.2 Loop performed by the use of hysteresis material.

4.2. Magnetorquers

4.2.1. Description of the system

Magnetorquers are essentially electromagnets, made by looping a wire multiple times with a known section, that generate a magnetic dipolar moment given by equation **4.2**.

$$\vec{\boldsymbol{\mu}} = \boldsymbol{n}\, \boldsymbol{I}\, \vec{\boldsymbol{A}}_{\boldsymbol{t}} \qquad \textbf{4.2}$$

The dipolar magnetic moment depends on the number of loops (n) of the wire, the intensity of the current run through it (I), and the area inside the loops (A_t); this area is considered as a vector, with a direction perpendicular to the surface in the standard way described in classical topology. This $\vec{\mu}$ is generally restricted (**4.3**) by the maximum power allowed to feed the actuator P_{max}.

$$\vec{\boldsymbol{\mu}}_{\boldsymbol{max}} = \boldsymbol{n}\ \vec{\boldsymbol{A}}_{\boldsymbol{t}} \sqrt{\frac{\boldsymbol{P_{max}}}{\boldsymbol{R_{wire}}}} \qquad \textbf{4.3}$$

where R_{wire} is the resistance to the current flow generated by the wire. In our study, we will use a commercial magnetorquer from New Space Systems (NSS), NCTR-M003 [11] that has a maximum power consumption of less than 250 mW, weight of under 30 g and a size of 72mm×15mm×13mm, values suitable for our mission, as three of them will be required. This system provides a maximum magnetic moment of 0.29 A m^2.

With those parameters and following from **4.3** we can deduce that the relation between $\vec{\boldsymbol{\mu}}$ and $\boldsymbol{P}$ is given by **4.4**

$$\boldsymbol{\mu} = \boldsymbol{n}\boldsymbol{A}_{\boldsymbol{t}} \sqrt{\frac{\boldsymbol{P}}{\boldsymbol{R_{wire}}}} = k\sqrt{\boldsymbol{P}} \qquad \textbf{4.4}$$

where k can be found by following **4.5**

$$k = \frac{\boldsymbol{\mu}}{\sqrt{\boldsymbol{P}}} = \frac{\boldsymbol{\mu_{max}}}{\sqrt{\boldsymbol{P_{max}}}} \qquad \textbf{4.5}$$

and the power consumed in every moment is given by **4.6**

$$\boldsymbol{P} = \sum \left(\frac{\boldsymbol{\mu_i}}{\boldsymbol{k}}\right)^2 \qquad \textbf{4.6}$$

with the index$i$ being used to identify the different magnetorquers.

4.2.2. Simulation details

In this case, the dipolar moment changes from one iteration to the next. To determine it, we will firstly calculate the target dipolar moment (as explained in section 3.6) that

would be required in order to stop completely the rotation. Then, we will check if the value is under the limits of the system; if, on the contrary, it is too large, the dipolar moment is forced to be the maximum available. Then, with this value the rotation rate evolution is modelled.

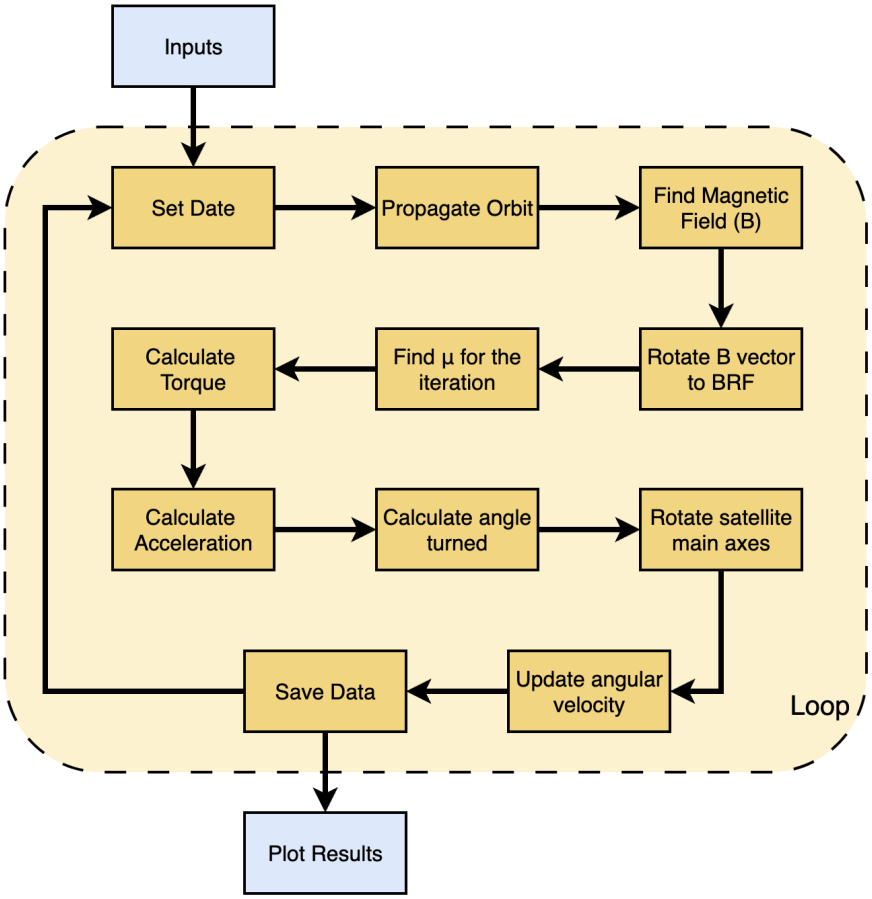

Figure 4.3 Loop performed by the use of magnetorquers.

Chapter 5

Code verification

A vital part of the modelling process is to verify the performance of the processes used. In this case as several methods will be compared, firstly it will be interesting to compare how both propagators used diverge for the same time variation; in this way, we can have a hindsight of which part of a potential difference can be attributable to the orbit propagators. Secondly, it will be necessary to compare the two magnetic models for several positions around the orbit. Lastly, the rotation of these fields should be monitored, in order to verify that the rotations are not modifying the value, just changing the reference system in which they are represented.

5.1. Propagator verification

In order to compare SGP4 and SGP8, the same real life TLE will be used to check the difference in behaviour. For this, only a reduced section of the code will be generated (included in the annexes), propagating the orbit for a total of 20 days, and storing a data point every 300s.

We will use the TLE from the Iridium 65 satellite on 16/05/1999 (**Figure 5.1**), which at that time orbited at a height of around 785 km over the Earth's surface.

```
1 25288U 98021D   99117.03750742 -.00000015  00000+0 -12318-4 0  1471
2 25288  86.3956 112.7366 0002518  67.8845 292.2618 14.34216487 55284
```

Figure 5.1 Iridium 65 TLE.

Plotting the absolute error for each direction, we can see that the difference is never larger than 0.4%, and most of the time remains below 0.05% **Figure 5.2.**

If we plot the error for the norm of the vector for the same set of data, we can see that the error always remains below 0.00018% (see **Figure 5.3**) which shows that in practice the difference is negligible.

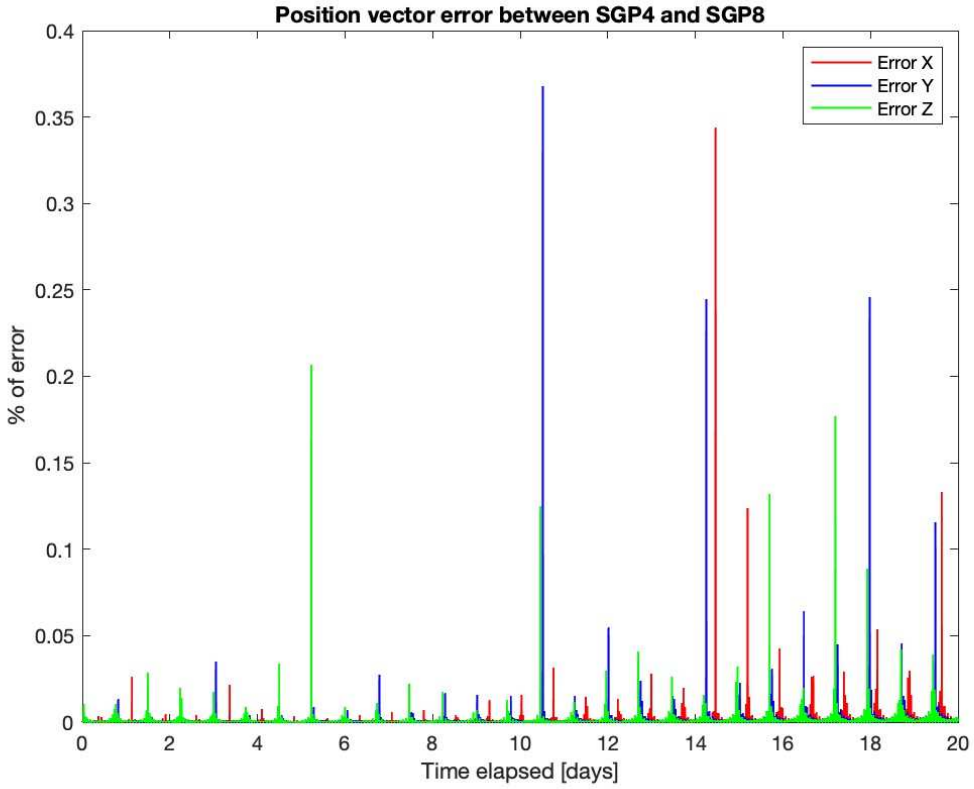

Figure 5.2 Propagated vector components error Iridium 65.

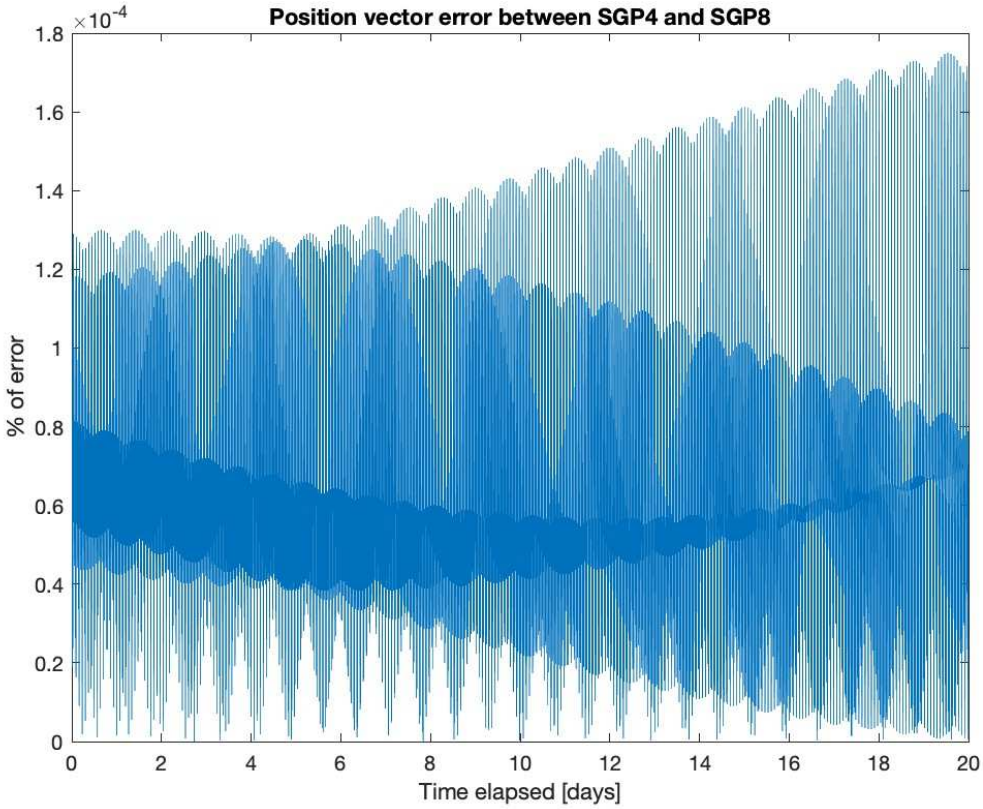

Figure 5.3 Propagated vector module error Iridium 65.

To check for possible altitude dependent issues, we select a TLE set from the lower altitude satellite Sateliot 1(**Figure 5.4**), then located at around 540 km of height. The data set corresponds to 18/01/2023.

```
1 47961U 21022AF  23018.66453903  .00029170  00000+0  16080-2 0  9999
2 47961  97.5171 280.0729 0019951 121.2971 239.0217 15.13768716 99338
```

Figure 5.4 Sateliot 1 TLE.

Plotting again the same both graphs, we can see that due to the atmospheric drag difference, there is a steady increase in the error form one to the other (**Figure 5.5** and **Figure 5.6**).

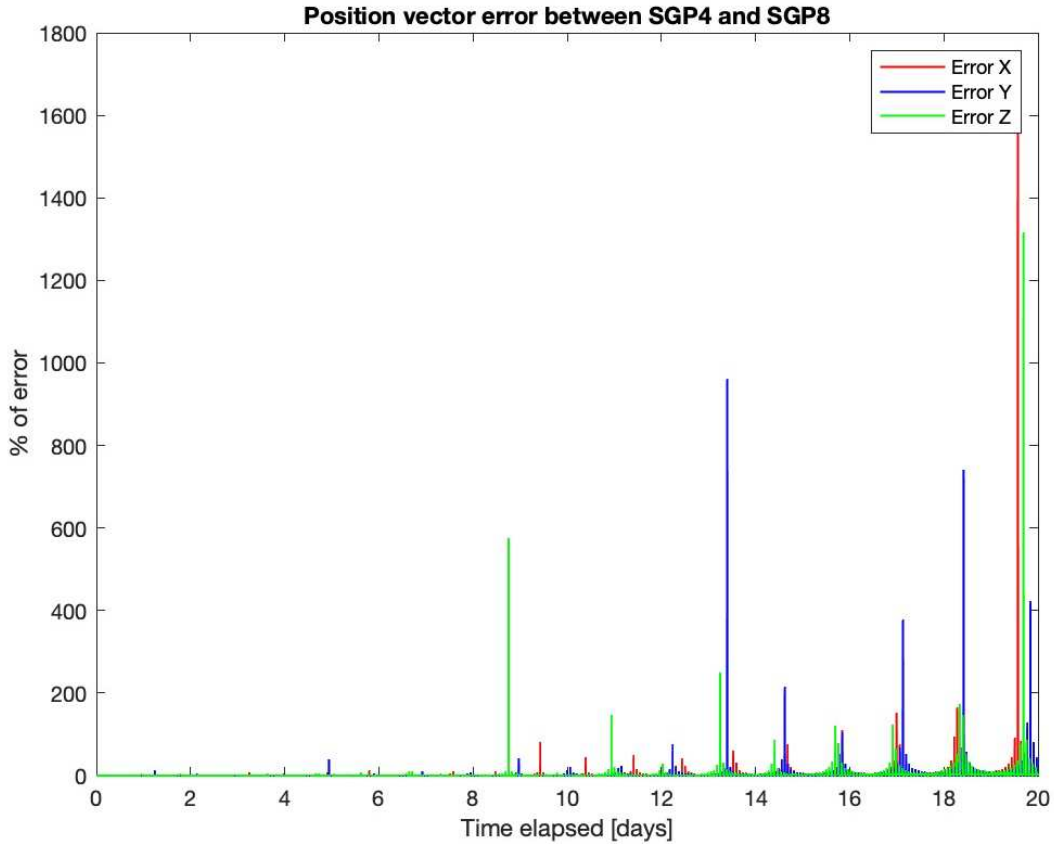

Figure 5.5 Propagated vector components error Iridium 65.

This is not unexpected, as TLEs should not be propagated for such long periods of time using SGP4 or SGP8. The inaccuracies in the TLEs, and the approximations in the very nature of SGPs will cause a rapid increase in the error. This is the reason why the NORAD publishes daily TLEs for all active satellites and the largest pieces of space debris.

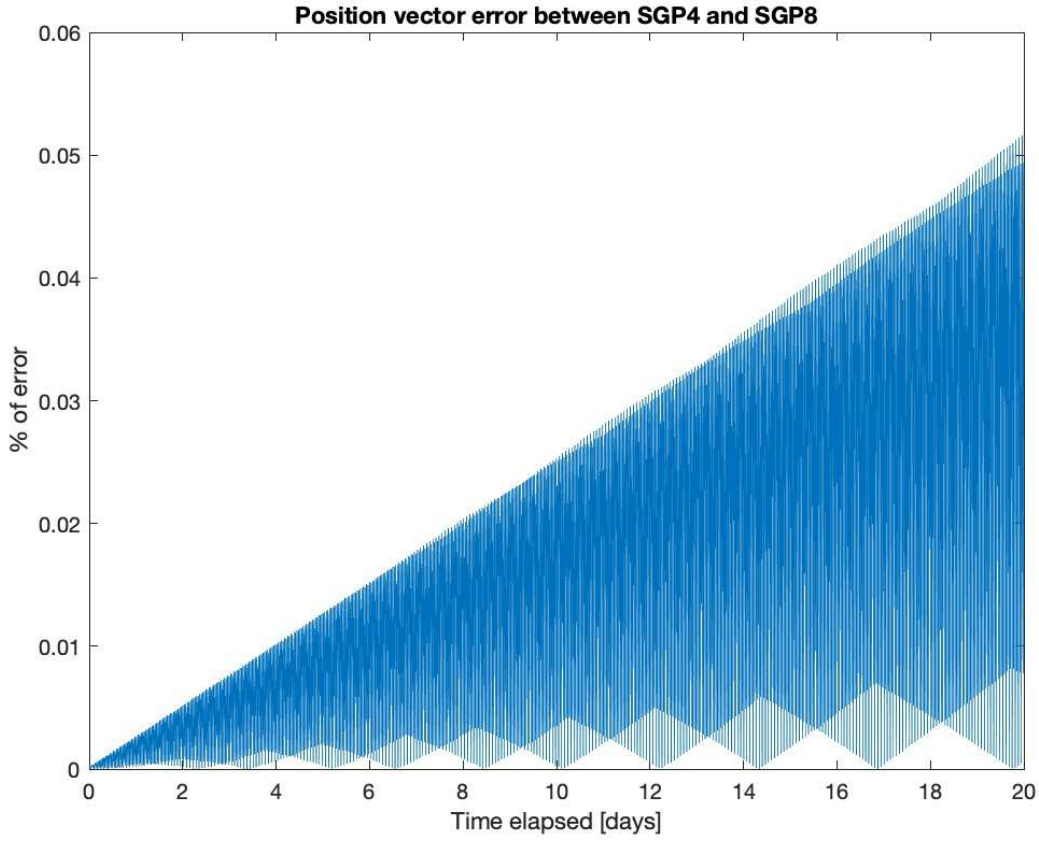

Figure 5.6 Propagated vector module error Iridium 65.

5.2. Magnetic model verification

Again, in order to compare the two magnetic models, IGRF and WMM, a reduced version (also included in the annexes) of the code has been made. This code will propagate the orbit from Sateliot 1 TLE using SGP8 during 20 days with a data point every 300s, and then will obtain the error in percentage between both magnetic models generated vectors. Here we do not need to worry about the possible errors in the position of the satellite; as our purpose is to compare the magnetic field models, we only need to determine the values predicted by these models in exactly the same point.

The comparison of the error component by component gives in some cases huge differences, all of them going in some cases beyond 50% and being the worst cases well over 300% in the component Y as shown in the **Figure 5.7**

Nevertheless, this can be attributed to the difference when component values are too small and a nonsignificant variation can lead to a large deviation in terms of percentage. In fact, when we plot the difference between their modules, the difference remains below 0.09% **Figure 5.8**.

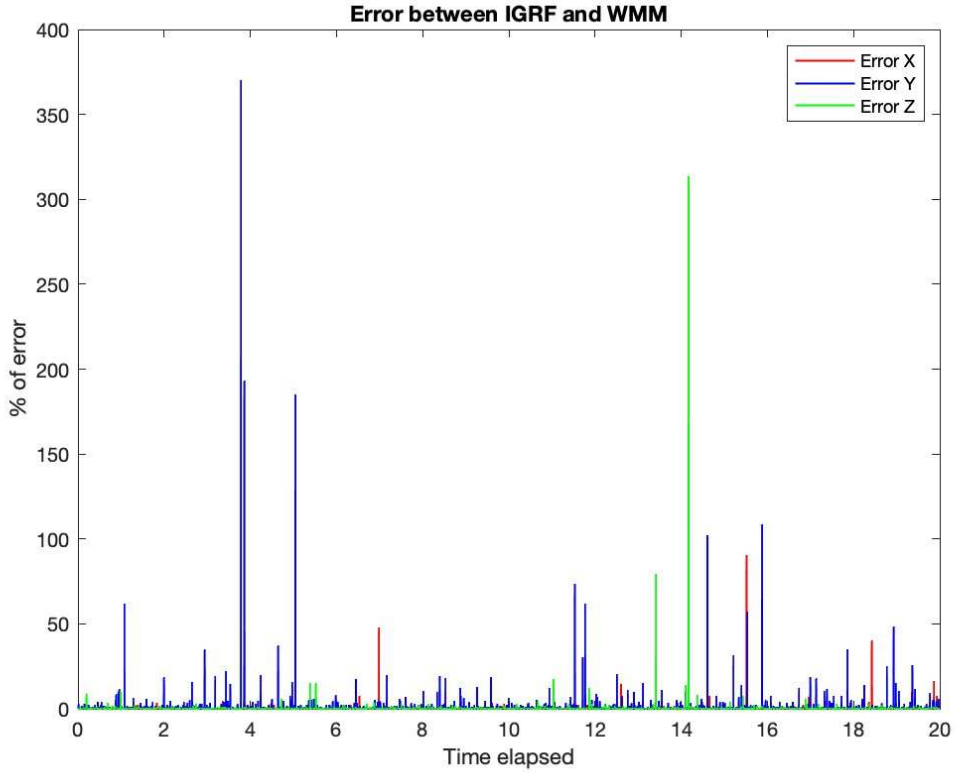

Figure 5.7 Magnetic field vector components error.

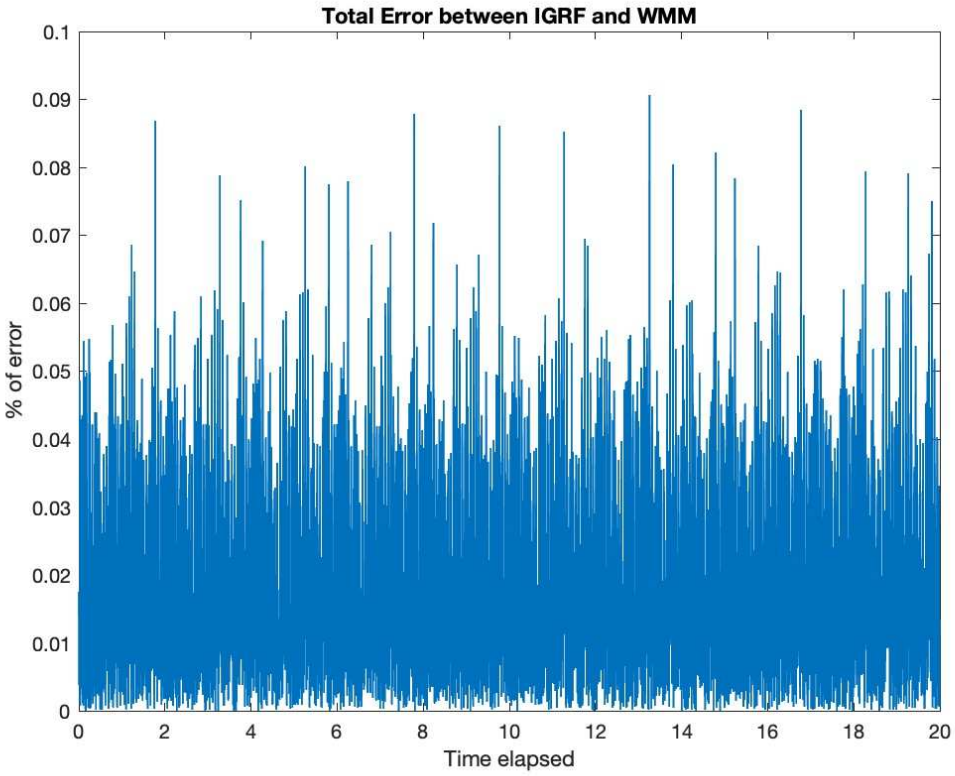

Figure 5.8 Magnetic field vector module error.

5.3. Rotation verification

The magnetic field is rotated three times, from NED to ECEF, from ECEF to ECI and from ECI to BRF. To verify these three rotations, the module of the vector rotated will be compared again using a simplified version code (included in the annexes). This code will only make use of SGP8 and IGRF, propagating the attitude and checking the error for each iteration; our baseline is Sateliot 1. In this case, as the simulation includes the update of the attitude, 1 second time steps will be performed for a total simulation time of 0.1 days to provide better results.

As seen in **Figure** 5.9, **Figure** 5.10 and **Figure** 5.11, the error of the three cases is in the order of magnitude of 10^{-16}, so low that can be dismissed as a truncation error from the software. Therefore, we can assume that rotations are performed adequately.

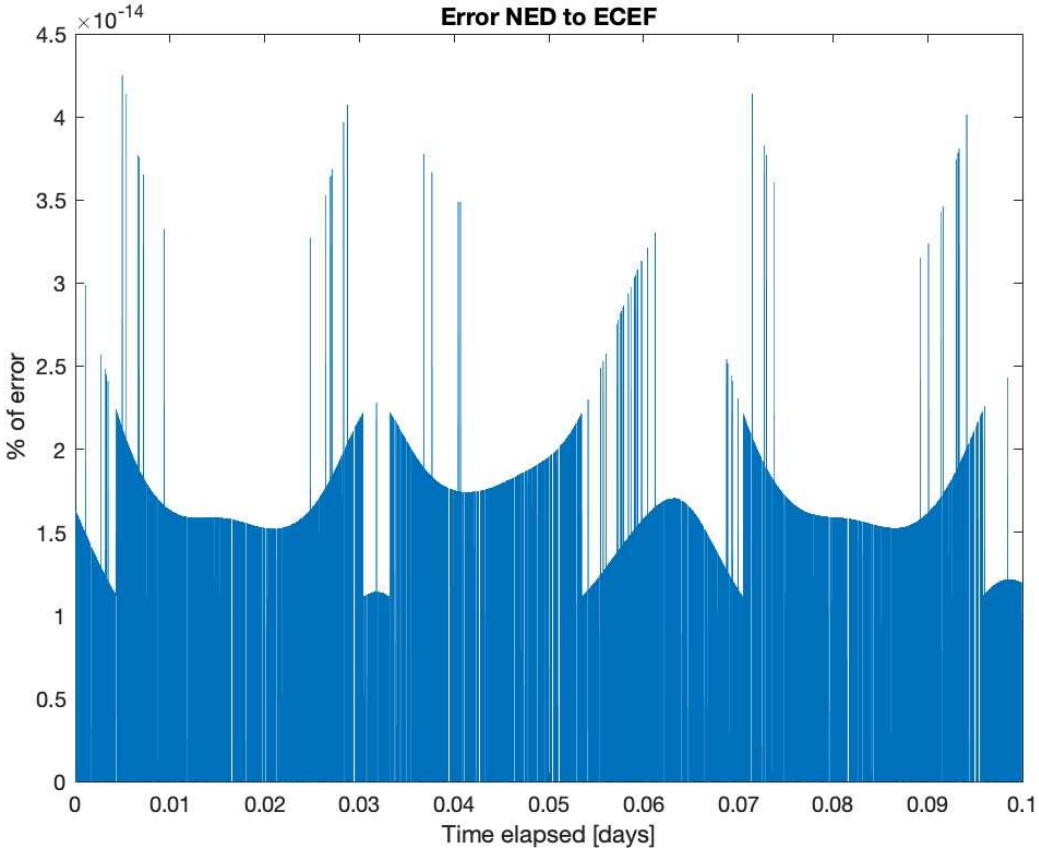

Figure 5.9 Module error from NED to ECEF.

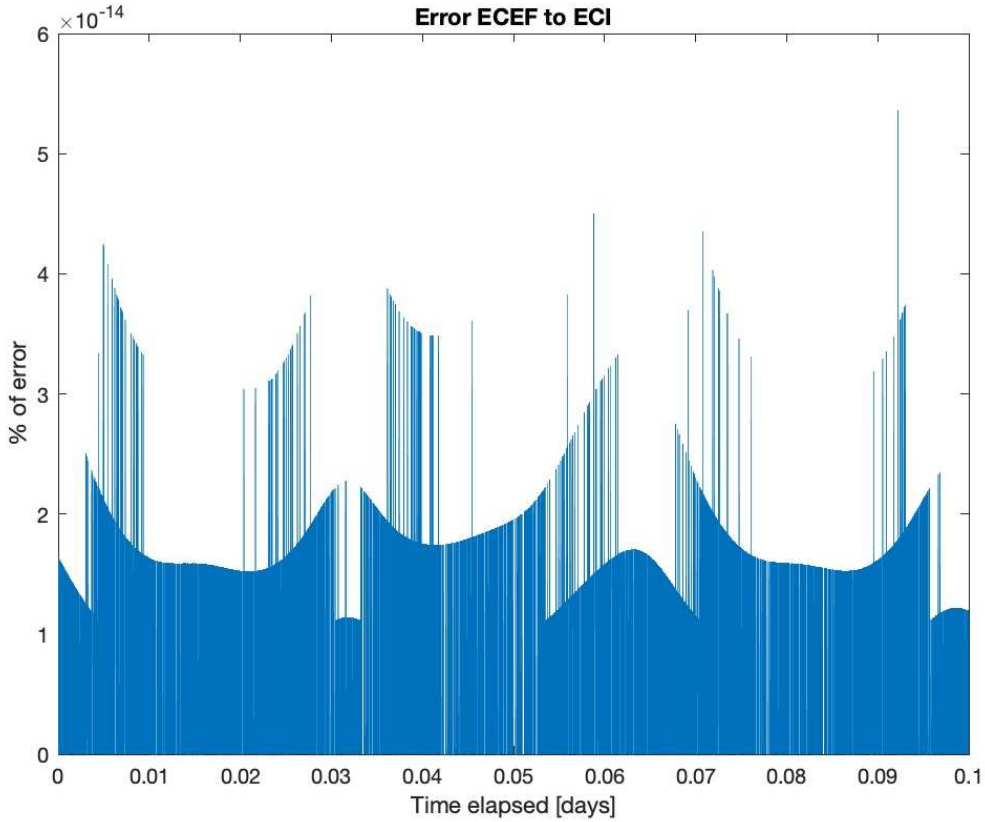

Figure 5.10 Module error from ECEF to ECI.

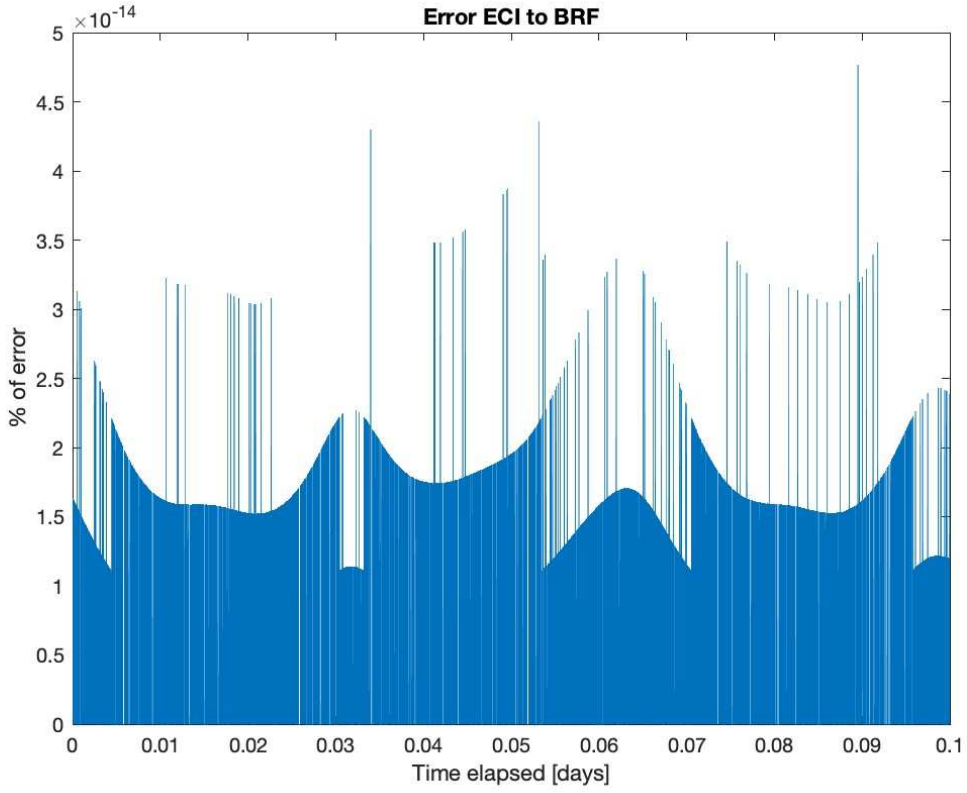

Figure 5.11 Module error from NED to ECEF.

Chapter 6

Magnetic ADCS Performance Simulations

6.1. Inputs

In order to undertake the simulations, three synthetic TLEs will be generated, differing between them mainly in the altitude of the orbit. These TLEs are shown in **Figure 6.1** for 500 km, **Figure 6.2** for 400 km and **Figure 6.3** for 300 km of altitude over mean sea level.

```
1 00000U 00000A   23031.67064815  .00000000  00000-0  38776-6 0    06
2 00000  95.0004   0.3102 0015914 272.7718 127.2973 15.23824759    07
```

Figure 6.1 Synthetic TLE for 500 km altitude orbit.

```
1 00000U 00000A   23031.67064815  .00000000  00000-0  19950-4 0    07
2 00000  94.9974   0.3041 0005341 308.8145 171.3603 15.55440026    07
```

Figure 6.2 Synthetic TLE for 400 km altitude orbit.

```
1 00000U 00000A   23031.67064815  .00000000  00000-0  11527-4 0    09
2 00000  95.0033   0.3092 0011684 271.1788 288.9117 15.94925733    04
```

Figure 6.3 Synthetic TLE for 300 km altitude orbit.

6.2. Results

Once the code has been run for both ADCS mechanisms, each one of them for the three TLEs stated in the previous section, several plots are generated to compare the results. If the plots are similar and the difference cannot be appreciated even when overlaid, only one of them will be shown.

6.2.1. Hysteresis

- 300 km: this simulation runs for 0.2 days with a time step of 2 seconds. Stabilization times are shown in **Figure 6.4**, and the rotation rate evolution is shown in **Figure 6.5**. We can easily notice that the rotation rate evolution is very similar for the four models, that overlay each other.

```
Command Window

iteration =

        7927

Satellite hysteresis stabilised after 0.1835 days. SGP4 WMM
Satellite hysteresis stabilised after 0.1835 days. SGP4 IGRF
Satellite hysteresis stabilised after 0.1835 days. SGP8 WMM
Satellite hysteresis stabilised after 0.1835 days. SGP8 IGRF
fx >>
```

Figure 6.4 Stabilisation times at 300 km altitude with hysteresis.

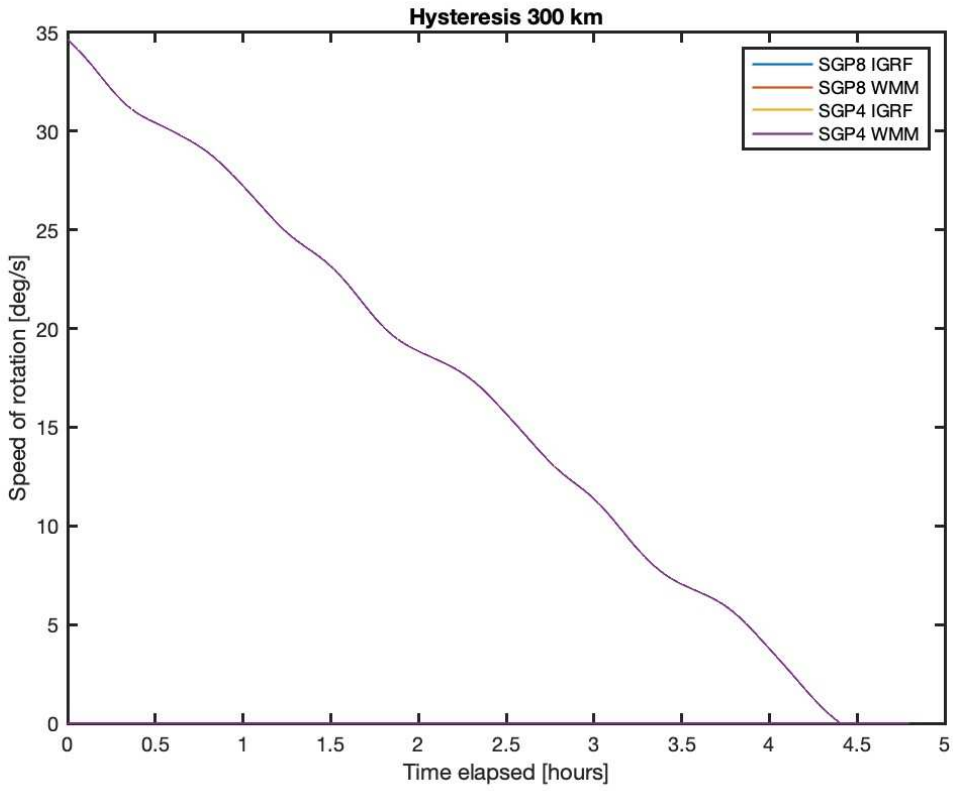

Figure 6.5 Rotation speed at 300 km altitude with hysteresis.

- 400 km: this simulation runs for 0.2 days with a step of 2 seconds. Now the stabilization times are shown in **Figure 6.6**, and the rotation rate evolution is shown in **Figure 6.7**. Again, the angular velocity evolution is undistinguishable for the four models.

```
Command Window

iteration =

        8391

Satellite hysteresis stabilised after 0.1942 days. SGP4 WMM
Satellite hysteresis stabilised after 0.1942 days. SGP4 IGRF
Satellite hysteresis stabilised after 0.1942 days. SGP8 WMM
Satellite hysteresis stabilised after 0.1942 days. SGP8 IGRF
fx >>
```

Figure 6.6 Stabilisation times at 400 km altitude with hysteresis.

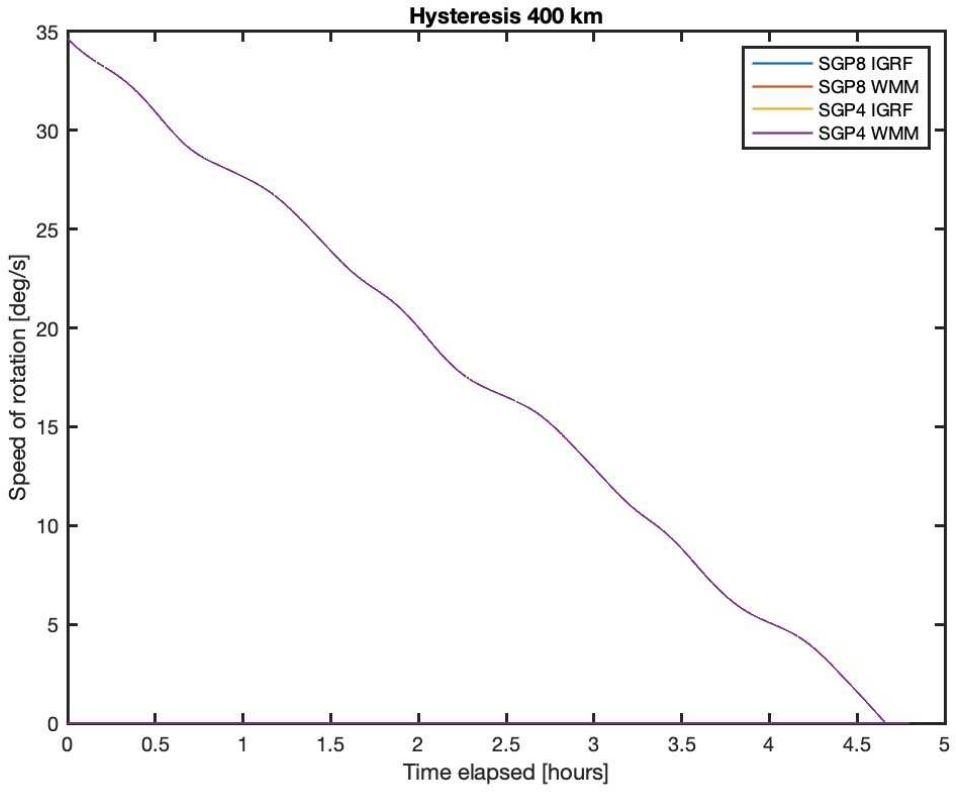

Figure 6.7 Rotation speed at 400 km altitude with hysteresis.

- 500 km: this simulation runs for 0.25 days with a time step of 2 seconds. The behaviour shown in **Figure** 6.8 and **Figure** 6.9 is completely analogous to the two cases already discussed.

```
Command Window

iteration =

        8716

Satellite hysteresis stabilised after 0.2017 days. SGP4 WMM
Satellite hysteresis stabilised after 0.2017 days. SGP4 IGRF
Satellite hysteresis stabilised after 0.2017 days. SGP8 WMM
Satellite hysteresis stabilised after 0.2017 days. SGP8 IGRF
fx >>
```

Figure 6.8 Stabilisation times at 500 km altitude with hysteresis.

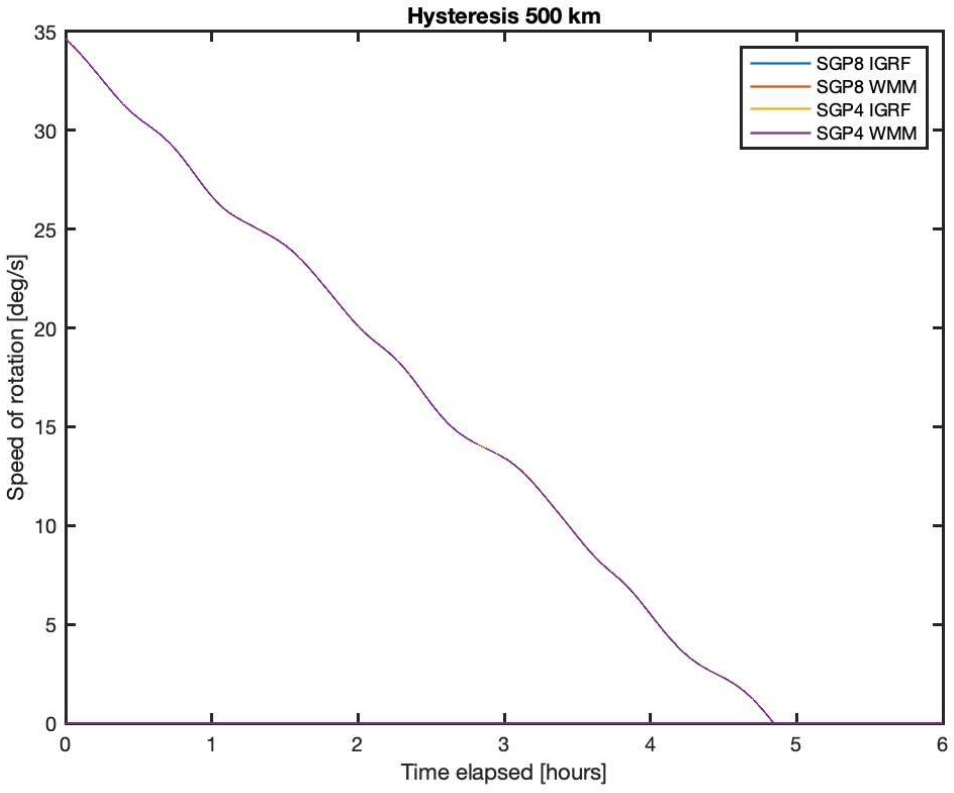

Figure 6.9 Rotation speed at 500 km altitude with hysteresis.

6.2.2. Magnetorquers

- 300 km: this simulation is run for 0.012 days with a time step of 1 second, obtaining **Figure 6.10**.

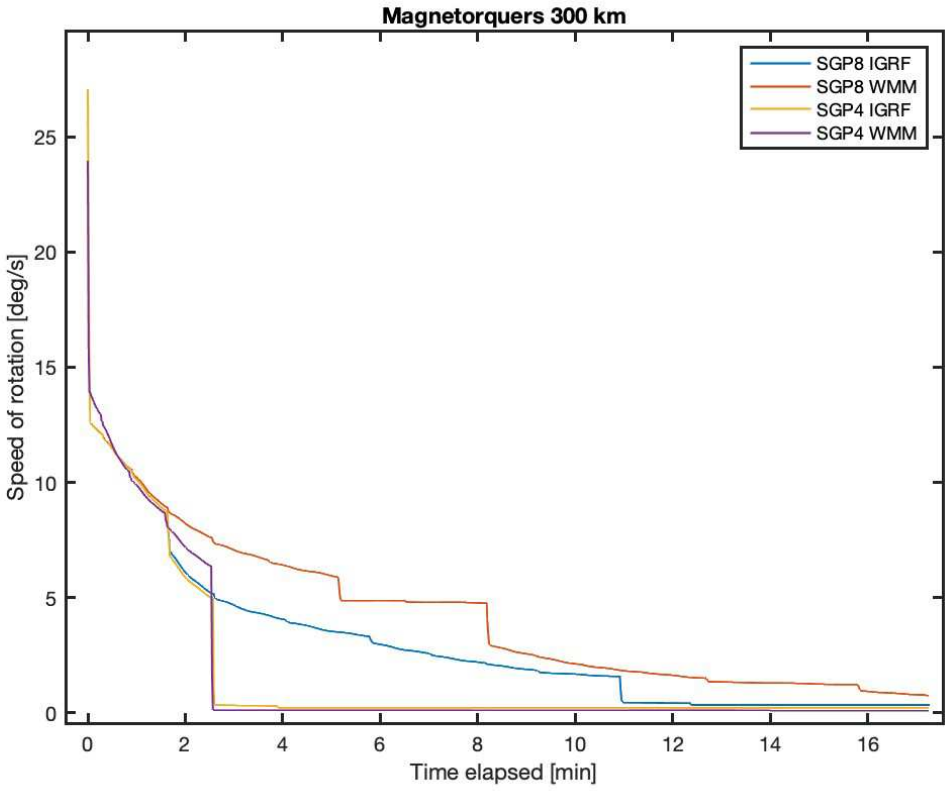

Figure 6.10 Rotation speed at 300 km altitude with magnetorquers.

- 400 km: this simulation is run for 0.03 days with a time step of 1 second, obtaining **Figure 6.11**.

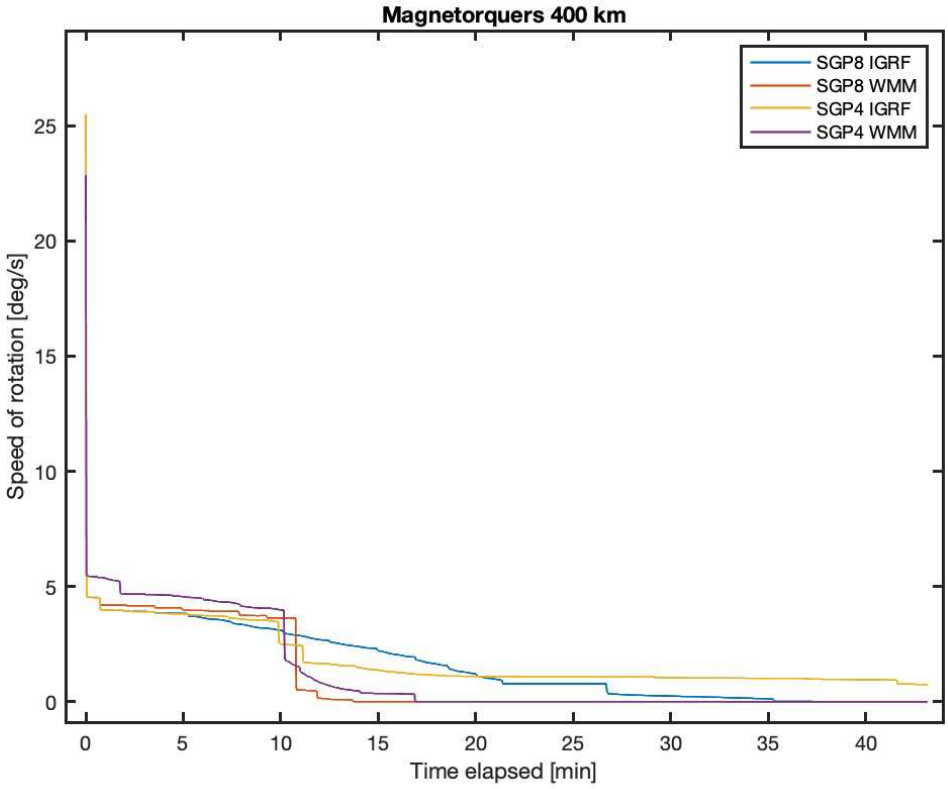

Figure 6.11 Rotation speed at 400 km altitude with magnetorquers.

- 500 km: this simulation is run for 0.02 days with a time step of 1 second **Figure 6.12**.

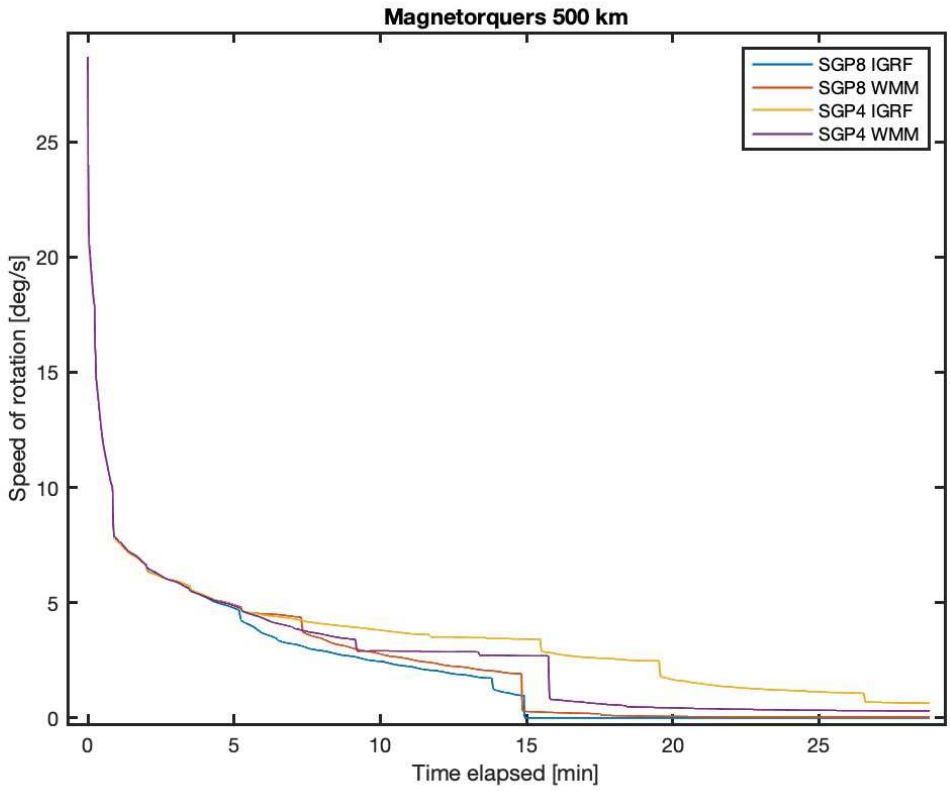

Figure 6.12 Rotation speed at 500 km altitude with magnetorquers.

6.3. Analysis of results

The results of the previous graphs are summarised in several tables, **Table 6.1** for the hysteresis rods and **Table 6.2** for the magnetorquers.

Table 6.1 Hysteresis rods results.

Altitude	Propagator	Magnetic model	Stabilization	Result
300 km	SGP4	WMM	4h 25min	There is no significant difference between the four different models at each height.
		IGRF		
	SGP8	WMM		
		IGRF		
400 km	SGP4	WMM	4h 40min	There is a decline in rotation speed whose steepness oscillates due to the variation in magnetic along the orbit.
		IGRF		
	SGP8	WMM		
		IGRF		

Table 6.1 Hysteresis rods results. (Cont.)

Altitude	Propagator	Magnetic model	Stabilization	Result
500 km	SGP4	WMM	4h 50min	As near the Earth the magnetic field is stronger, the dampening effect is also more significant.
		IGRF		
	SGP8	WMM		
		IGRF		

Table 6.2 Magnetorquer results.

Altitude	Propagator	Magnetic model	Stabilization	Result
300 km	SGP4	WMM	26 min	For the first 2 s all plots are the same.
		IGRF	26 min	
	SGP8	WMM	16 min	
		IGRF	11 min	
400 km	SGP4	WMM	11 min 25 s	All except SGP4 WMM are the same for 42 s.
		IGRF	37 min	
	SGP8	WMM	10 min 50 s	
		IGRF	21 min	
500 km	SGP4	WMM	15 min 50 s	All are equal for the first 54 s.
		IGRF	26 min 30 s	
	SGP8	WMM	14 min 50 s	
		IGRF	15 min	

In general, it can be seen that hysteresis rods take longer than magnetorquers to stabilize the satellite, a result in line with our expectations. All the magnetorquer graphs have a very steep decrease in rotation speed at the beginning, that finally gets the satellite to speeds of under 1 degree per second in less than an hour, and in most cases under 20 minutes.

The power consumption of the magnetorquers can be calculated as stated in equation **4.5** and plotted for each one of the simulations.

For the 300km case, **Figure 6.13** is produced (the first peak is cropped out to make the rest of the graph noticeable). In this case there is a large 250mW peak at the beginning, and then several smaller ones that consume less than 25 mW.

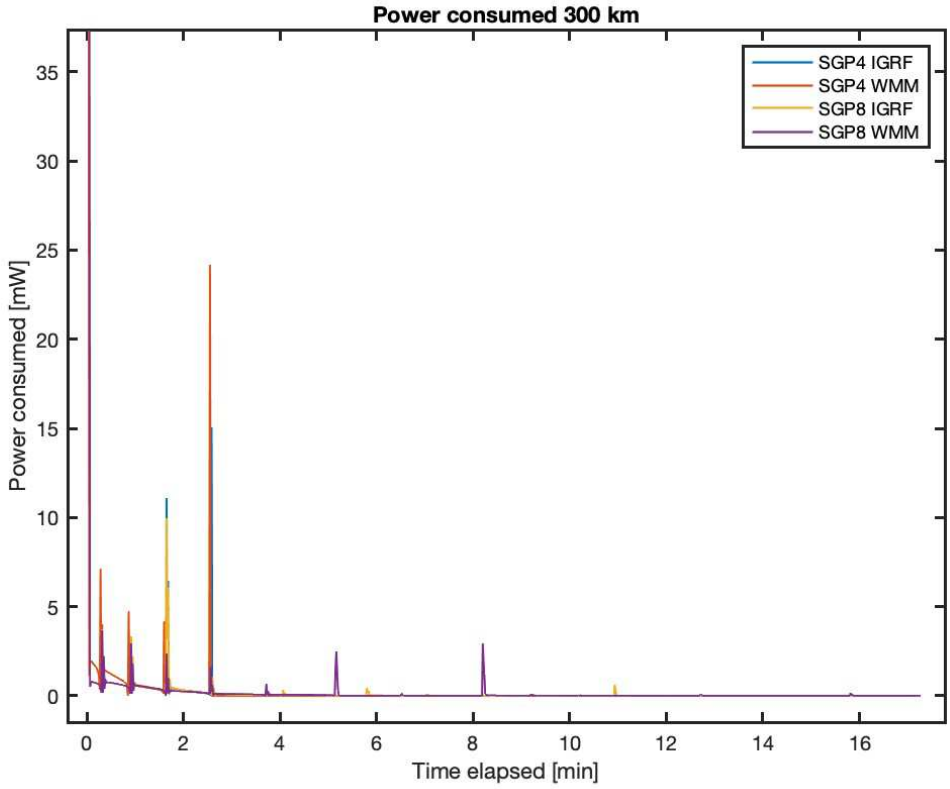

Figure 6.13 Power used during stabilization in the 300 km altitude orbit.

For the 400 km case we obtain **Figure 6.14** (again, the first peak is cropped out to make the rest of the graph noticeable); now the peak is smaller than in the previous case, being just a 157mW peak. Further on in the simulation, there appear several smaller peaks of less than 20 mW. The rest of the values will only consume less than 5 mW.

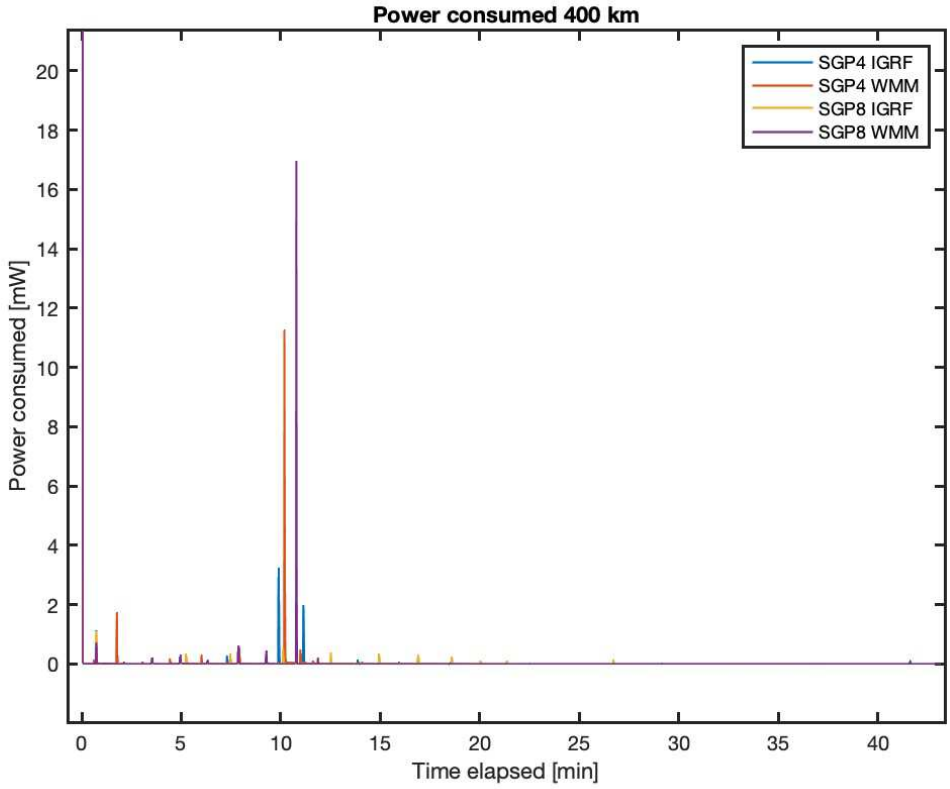

Figure 6.14 Power used at the400 km altitude orbit.

For the 500 km case, as seen in **Figure6.15**, the early peak is just 67 mW high (nevertheless, this first peak is cropped out to make the rest of the graph noticeable), Secondary peaks are far smaller, with a maximum 25 mW peak at 48 seconds, and several peaks of less than 5 mW

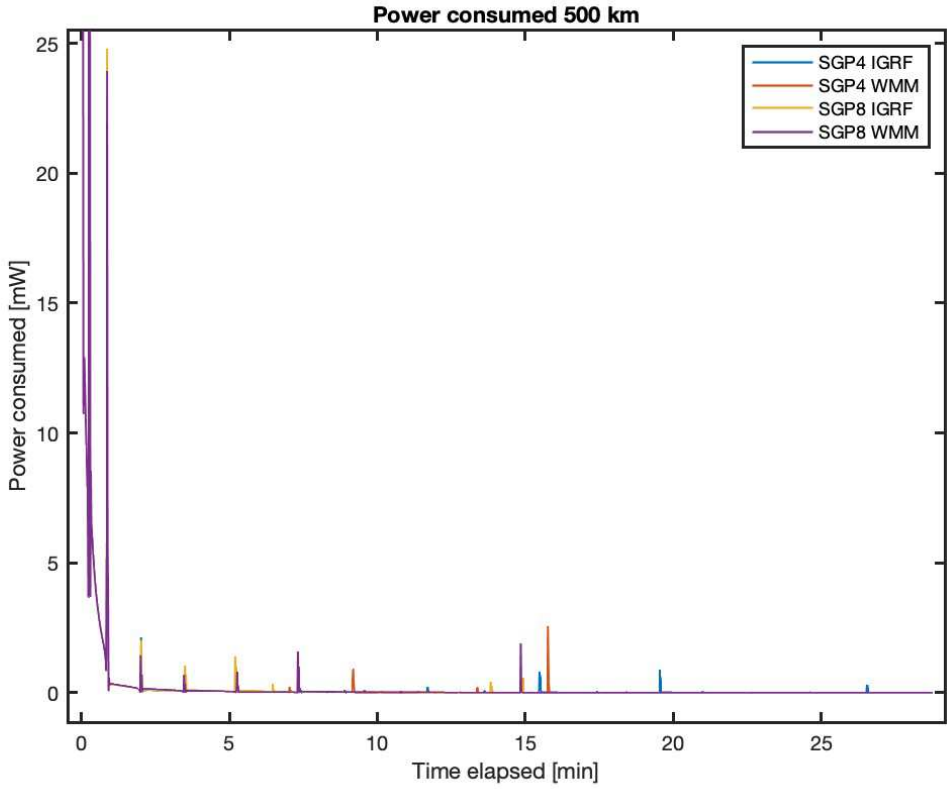

Figure6.15 Power consumed 500 km altitude.

Figure 6.13, **Figure** 6.14 and **Figure6.15** can be summarised in **Table 6.3**.

Table 6.3 Power consumption of each simulation.

Altitude	Biggest peak	Residual	Analysis
300 km	250 mW	<1 mW	Has a very large peak of 250 mW and other 4 smaller ones of less than 25 mW, but in general the power consumption is low.
400 km	157 mW	<1 mW	Has a large peak of 157 mW and a couple more of less than 10 mW, but in general the power consumption is low.
500 km	67 mW	<1 mW	Has a larger peak of 67 mW other one of 24 m-W and a several bellow 5 mW, but in general the power consumption is low.

Regarding the requirements, the **Table 6.4** analyses the compliance of both systems of the requirements applicable to the subsystem as studied in this document.

Table 6.4 ADCS Requirement compliance.

Code	Description	Compliance Hysteresis	Compliance Magnetorquers
R-6	The ADCS system shall have a mass below 100 grams.	Yes (<45 g)	Yes (<3x30 g)
R-7	The ADCS shall fit inside the satellite	Yes (elongated, 50 mm longest dimension)	Yes (elongated, 70 mm longest dimension)
R-8	The ADCS subsystem shall maintain at all times the angular velocity below 2 degrees per second in all axis.	Yes	Yes
R-9	The ADCS subsystem shall detumble the satellite in a period no longer than seven days.	Yes* (4-5 h)	Yes (< 1 h)
R-10	An active system shall control the attitude of the satellite within a maximum error of 5 degrees.	Not applicable	Yes
R-11	The ADCS system shall have a power consumption below 250 mW.	Yes (0 mW)	Yes (<250 mW)
R-13	The ADCS subsystem shall fulfill the mission requirements at all times (including eclipse periods).	Yes	Yes

* Although compliant has a significant undetermined error due to the linearity assumptions of done in the calculation of the coercivity.

Both systems comply with all of them, nevertheless, the magnetorquers although slightly more massive, demanding power and requiring active software control to operate, perform the detumbling much faster than the hysteresis, can be deactivated to reduce their effect on the measurements and can hold a desired attitude in case of need, therefore being a much better candidate for the mission. Magnetorquers can be also downsized, to avoid such large peaks of power consumption, mass, and still provide better results than the hysteresis rods.

Chapter 7

Conclusions and future work

7.1. Conclusions

During the execution of this project, a code to model the two desired ADCS has been developed as shown in Chapter 3 and Chapter 4, verified as seen in Chapter 5and used for the desired orbits and characteristics as seen in Chapter 6.

As seen in section 6.3 magnetorquers and hysteresis rods both achieve rotational levels under 1 deg/sec in under a day. Nevertheless, the most efficient one to stabilise the satellite are the magnetorquers, as they achieve the steady state in less than 30minutes, against the 4 to 5 hours required by the hysteresis rods.

Additionally, both methods are compliant with the requirements in terms of size, mass and peak power consumption, as seen in Table 6.4. In this case the hysteresis rods are more favourable in those regards. But the fact that the magnetorquers perform better in those requirements related to performance, and the fact that they have other advantages like being able to select a desired attitude and the option of being disconnected to avoid interfering with the measurements of the satellite, make them the better choice for our satellite outweighs the peak power consumption produced by the magnetorquers.

Another disadvantage of hysteresis rods is that, if some magnetization remains, the associated parasitic magnetic dipolar moment would become a source of attitude noise (excursions). This is not the case with magnetorquers, which again seem the best choice for our mission.

7.2. Future work

As with any project, the work is not perfect nor complete, and can always be improved and refined, both to include more cases of ADCS systems, to improve computational efficiency, and to simplify its use, among other possible improvement

Regarding the extra cases, the joint use of hysteresis rods and a permanent magnet has not been tackled and may be an interesting solution to study further down the line. Also, as mentioned in section2.1, the magnetic models used (IGRF 13 and WMM-2020) are both only suitable for the 2020-2025 interval, so the code should be adapted to be used outside that range.

Said code can also be reviewed with the purpose of reducing computational burden and memory used in an attempt to improve overall efficiency, and to avoid useless

iterations by verifying stabilisation with respect to a threshold rotational speed for all methods, and stopping the loop once it is reached.

Finally, to improve the user experience, a graphic user interface could be implemented to provide a friendlier working environment, where it should include places to add the data related to the satellite, select the models used for the simulation, select the ADCS method and select the TLE file.

Bibliography

[1]Buis.,A., "Earth's Atmosphere: A Multi-layered Cake – Climate Change: Vital Signs of the Planet". Climate Change: Vital Signs of the Planet. https://climate.nasa.gov/news/2919/earths-atmosphere-a-multi-layered-cake/ (accessed on 1st of february2023).

[2]Mahooti., M., "SGP4". MathWorks - Makers of MATLAB and Simulink - MATLAB & Simulink. https://www.mathworks.com/matlabcentral/fileexchange/62013-sgp4 (accessed on1st offebruary2023).

[3]Mahooti., M., "SGP8". MathWorks - Makers of MATLAB and Simulink - MATLAB & Simulink. https://www.mathworks.com/matlabcentral/fileexchange/75492-sgp8 (accessed 1st offebruary2023).

[4]"MATLAB - El lenguaje del cálculo técnico". MathWorks - Creadores de MATLAB y Simulink - MATLAB y Simulink - MATLAB & Simulink. https://es.mathworks.com/products/matlab.html (accessed 1stoffebruary2023).

[5]"Aerospace toolbox". MathWorks - Makers of MATLAB and Simulink - MATLAB & Simulink. https://se.mathworks.com/products/aerospace-toolbox.html (accessed 1 of february de 2023).

[6]"Mapping toolbox". MathWorks - Creadores de MATLAB y Simulink - MATLAB y Simulink - MATLAB & Simulink. https://es.mathworks.com/products/mapping.html (accessed st1 of february2023).

[7]Alken, P., Thébault, E., Beggan, C.D. et al. International Geomagnetic Reference Field: the thirteenth generation. Earth Planets Space 73, 49 (2021). https://doi.org/10.1186/s40623-020-01288-x

[8]Chulliat, A., Brown; W., Alken, P.,Beggan C.D.;et al.;"The US/UK World Magnetic Model for 2020-2025" (2020) https://doi.org/10.25923/ytk1-yx3

[9]"Quaternion - Encyclopedia of Mathematics". Encyclopedia of Mathematics. https://encyclopediaofmath.org/index.php?title=Quaternion (accessed 1st of february 2023).

[10]Sols, A., “Requirements Engineering and Management”, Cap. 5,*Systems Engineering: theory and practice*,p. 135,Universidad Pontificia Comillas, Madrid (2014).

[11]A. Farrahi y Á. Sanz-Andrés, "Efficiency of Hysteresis Rods in Small Spacecraft Attitude Stabilization", The Scientific World Journal, vol. 2013, pp. 1–17, 2013. (accessed 1st offebruary de 2023). https://doi.org/10.1155/2013/459573

[12]"NCTR-M003 Magnetorquer Rod | satsearch". The global marketplace for the space industry.
https://satsearch.co/products/newspace-systems-nctr-m003-magnetorquer-rod
(accessed 1st of february de 2023).

Annexes

Main code

```
%-------------------------------------------------------------------------
%                                  TFM
%
%-------------------------------------------------------------------------
% This code propagates the attitude for a satellite using magnetic control
% given an initial TLE file.
% Uses SGP4 and SGP8 as orbit propagators, and models Earth's magnetic
% field by means of IGRF and WMM models.
% With those models generates 4 predictions for the rotation speed
% evolution with the system selected
%-------------------------------------------------------------------------
clc
clear
format long g
%-------------------------------------------------------------------------
%---------------------------------INPUTS----------------------------------
%-------------------------------------------------------------------------
% Select the type of actuators:
type=2;                              % 1 means hysteresis rods
                                     % 2 means magnetorquers

% TLE files names
fname = 'Final_Orbit_500km.txt';
% Tensor of Inertia [g*mm^2]
I=[[37971.2,-17.4,-15.7];
   [-17.4,37596.6,238.8];
   [-15.7,238.8,37727.4]];

% Initial rotation speed
rotspeed14=[1,1,1]*deg2rad(20);      % [rad/s] SGP4 WMM
rotspeed24=[1,1,1]*deg2rad(20);      % [rad/s] SGP4 IGRF
rotspeed18=[1,1,1]*deg2rad(20);      % [rad/s] SGP8 WMM
rotspeed28=[1,1,1]*deg2rad(20);      % [rad/s] SGP8 IGRF

% Propagation parameters
propdays=1;                         % propagation time [days]
step_sec = 1;                        % step [seconds]

% Hysteresis rods parameters
Hc=17.27;                            % Coercivity [A/m]
V=5.e-6;                             % Volume of the magnetic rod [m3]
Bsat=1.53;                           % Saturation magnetic field [T]
Chyst=(4*V*Hc*step_sec)/(360*Bsat); % Constant for hysteresis calculations

% Magnetorquer parameters
mupmax=0.29;                         % Max dipolar moment [A*m^2]
Pmax=250;                            % Max power used
powrconst=(mupmax)/sqrt(Pmax);       % Relation between Pmax and mumax
mumax=powrconst*sqrt(150);

% Satellite main axis at ECI frame at t=0
Sx14=[1,0,0]; Sy14=[0,1,0]; Sz14=[0,0,1];  % SGP4 WMM
Sx24=[1,0,0]; Sy24=[0,1,0]; Sz24=[0,0,1];  % SGP4 IGRF
Sx18=[1,0,0]; Sy18=[0,1,0]; Sz18=[0,0,1];  % SGP8 WMM
Sx28=[1,0,0]; Sy28=[0,1,0]; Sz28=[0,0,1];  % SGP8 IGRF
```

```
%-------------------------------------------------------------------------
%-------------------------Get initial parameters--------------------------
%-------------------------------------------------------------------------
% Reference vector for the ECI frame coordinate system
Inertialframe=[1,1,1]/(sqrt(3));

% Obtain orbit data form TLE
[satdata]= Orbitalp (fname);

% Change Tensor of Inertia in units from [g*mm^2] to [kg*m^2]
I=I/(10^9);                 % [kg*m^2]

% Time parameters for the propagation of satellite's state vector
% forward (+) or backward (-) [minutes]
tsince = propdays*1440;     % Propagation time [minutes]
step=step_sec/60;           % Step [minutes]
t=0;                        % First step
len=fix(tsince/step);       % Number of iterations

%----------------------Generate the storage arrays-----------------------

% SGP4 WMM rotation speed storage
omega14X(len)=0;            %Storage for X value
omega14Y(len)=0;            %Storage for Y value
omega14Z(len)=0;            %Storage for Z value
omega14M(len)=0;            %Storage for module
% SGP4 IGRF rotation speed storage
omega24X(len)=0;            %Storage for X value
omega24Y(len)=0;            %Storage for Y value
omega24Z(len)=0;            %Storage for Z value
omega24M(len)=0;            %Storage for module
% SGP8 WMM rotation speed storage
omega18X(len)=0;            %Storage for X value
omega18Y(len)=0;            %Storage for Y value
omega18Z(len)=0;            %Storage for Z value
omega18M(len)=0;            %Storage for module
% SGP8 IGRF rotation speed storage
omega28X(len)=0;            %Storage for X value
omega28Y(len)=0;            %Storage for Y value
omega28Z(len)=0;            %Storage for Z value
omega28M(len)=0;            %Storage for module

% Power consumed storage
powerc14(len)=0;            % SGP4 WMM
powerc24(len)=0;            % SGP4 IGRF
powerc18(len)=0;            % SGP8 WMM
powerc28(len)=0;            % SGP8 IGRF

% Store time of the iteration
timed(len)=0;
timeh(len)=0;
timem(len)=0;

% Stabilisation date storage
stabilization(4)=0;

% Iteration counter initialization
iteration=1;
```

```
%-------------------------Get the beginning year-------------------------
if (satdata.year< 57)
   year0 = satdata.year + 2000;
else
   year0 = satdata.year + 1900;
end

%------------------------------------------------------------------------
%-----------------------------------Loop---------------------------------
%------------------------------------------------------------------------
while t<tsince
%-----------------------------Time update--------------------------------
   % Calculate date for current iteration
   if satdata.doy+(t/1440) < 365
       doy = (satdata.doy+(t/1440));
       year=year0;
   else
       doy = (satdata.doy+(t/1440)-365);
       year=year0+1;
   end

   % Calculate date parameters
   [mon,day,hr,minute,sec] = days2mdh(year,doy);
   % Date as decimal year
   dy = decyear(year,mon,day,hr,minute,sec);
   % Generate UTC vector
   utc=[year,mon,day,hr,minute,sec];

%-------------------------Propagate the orbit----------------------------
   [rteme4, vteme4] = sgp4(t, satdata);        % SGP4 [km, km/s]
   [rteme8, vteme8] = sgp8(t, satdata);        % SGP8 [km, km/s]
% Convert position vector units from [km] to [m]
   rteme4=rteme4*1000;                         % [m]
   rteme8=rteme8*1000;                         % [m]

% Calculate LLA coordinates SGP4
   lla4=eci2lla([rteme4(1),rteme4(2),rteme4(3)],utc);
   lat4=lla4(1);     lon4=lla4(2);    h4=lla4(3);
% Calculate LLA coordinates SGP8
   lla8=eci2lla([rteme8(1),rteme8(2),rteme8(3)],utc);
   lat8=lla8(1);    lon8=lla8(2);    h8=lla8(3);

%-----------------------Calculate magnetic fields------------------------
% SGP4 [nT]
   [XYZ14, H14, D14, I14, F14] = wrldmagm(h4,lat4,lon4,dy,'2020'); % WMM
   [XYZ24,H24,D24,I24,F24] = igrfmagm(h4,lat4,lon4,dy,13);         % IGRF
   XYZ24=transpose(XYZ24);         % Adapt magnetic field vector format
% SGP8  [nT]
   [XYZ18, H18, D18, I18, F18] = wrldmagm(h8,lat8,lon8,dy,'2020'); % WMM
   [XYZ28,H28,D28,I28,F28] = igrfmagm(h8,lat8,lon8,dy,13);         % IGRF
   XYZ28=transpose(XYZ28);         % Adapt magnetic field vector format

%--------------------Transform magnetic fields into BRF------------------
%..............................NED to ECEF.............................
% SGP4 WMM   [nT]
   [ECEF14(1),ECEF14(2),ECEF14(3)]=ned2ecefv(XYZ14(1),XYZ14(2),XYZ14(3),lat
4,lon4);
% SGP4 IGRF  [nT]
   [ECEF24(1),ECEF24(2),ECEF24(3)]=ned2ecefv(XYZ24(1),XYZ24(2),XYZ24(3),lat
4,lon4);
```

```
% SGP8 WMM   [nT]
   [ECEF18(1),ECEF18(2),ECEF18(3)]=ned2ecefv(XYZ18(1),XYZ18(2),XYZ18(3),lat
8,lon8);
% SGP8 IGRF  [nT]
   [ECEF28(1),ECEF28(2),ECEF28(3)]=ned2ecefv(XYZ28(1),XYZ28(2),XYZ28(3),lat
8,lon8);
%.............................ECEF to ECI.............................
   eci14=ecef2eci(utc,ECEF14);     % SGP4 WMM   [nT]
   eci24=ecef2eci(utc,ECEF24);     % SGP4 IGRF  [nT]
   eci18=ecef2eci(utc,ECEF18);     % SGP8 WMM   [nT]
   eci28=ecef2eci(utc,ECEF28);     % SGP8 IGRF  [nT]
%.................Quaternion for ECI to BRF rotation.................
% SGP4 WMM
   S14=Sx14+Sy14+Sz14;             % Generate satellite orientation vector
   S14=S14/norm(S14);              % Normalise orientation vector
   pv14=cross(Inertialframe,S14);  % Calculate rotation axis
   % Calculate rotation angle
   if S14(1)+S14(2)+S14(3)<0
       fi14=deg2rad(180)-
(asin(norm(pv14))/(norm(Inertialframe)*norm(S14)));
   else
       fi14=asin(norm(pv14)/(norm(Inertialframe)*norm(S14)));
   end
   % Angle to reverse rotation
   antifi14=-fi14;
   % Calculate quaternion scalar part
   p014=cos(fi14/2);
   antip014=cos(antifi14/2);
   % Calculate quaternion vector part
   if norm(pv14)<0.0000001
       p14=transpose((pv14)*0);
       antip14=p14;
   else
       p14=transpose((pv14/norm(pv14))*sin(fi14/2));
       antip14=transpose((pv14/norm(pv14))*sin(antifi14/2));
   end

% SGP4 IGRF
   S24=Sx24+Sy24+Sz24;             % Generate satellite orientation vector
   S24=S24/norm(S24);              % Normalise orientation vector
   pv24=cross(Inertialframe,S24);  % Calculate rotation axis
   % Calculate rotation angle
   if S24(1)+S24(2)+S24(3)<0
       fi24=deg2rad(180)-
(asin(norm(pv24))/(norm(Inertialframe)*norm(S24)));
   else
       fi24=asin(norm(pv24)/(norm(Inertialframe)*norm(S24)));
   end
   % Angle to reverse rotation
   antifi24=-fi24;
   % Calculate quaternion scalar part
   p024=cos(fi24/2);
   antip024=cos(antifi24/2);
   % Calculate quaternion vector part
   if norm(pv24)<0.0000001
       p24=transpose((pv24)*0);
       antip24=p24;
   else
       p24=transpose((pv24/norm(pv24))*sin(fi24/2));
       antip24=transpose((pv24/norm(pv24))*sin(antifi24/2));
   end
```

```
% SGP8 WMM
   S18=Sx18+Sy18+Sz18;              % Generate satellite orientation vector
   S18=S18/norm(S18);               % Normalise orientation vector
   pv18=cross(Inertialframe,S18);  % Calculate rotation axis
   % Calculate rotation angle
   if S18(1)+S18(2)+S18(3)<0
       fi18=deg2rad(180)-
(asin(norm(pv18))/(norm(Inertialframe)*norm(S18)));
   else
       fi18=asin(norm(pv18)/(norm(Inertialframe)*norm(S18)));
   end
   % Angle to reverse rotation
   antifi18=-fi18;
   % Calculate quaternion scalar part
   p018=cos(fi18/2);
   antip018=cos(antifi18/2);
   % Calculate quaternion vector part
   if norm(pv18)<0.0000001
       p18=transpose((pv18)*0);
       antip18=p18;
   else
       p18=transpose((pv18/norm(pv18))*sin(fi18/2));
       antip18=transpose((pv18/norm(pv18))*sin(antifi18/2));
   end

% SGP8 IGRF
   S28=Sx28+Sy28+Sz28;              % Generate satellite orientation vector
   S28=S28/norm(S28);               % Normalise orientation vector
   pv28=cross(Inertialframe,S28);  % Calculate rotation axis
   % Calculate rotation angle
   if S28(1)+S28(2)+S28(3)<0
       fi28=deg2rad(180)-
(asin(norm(pv28))/(norm(Inertialframe)*norm(S28)));
   else
       fi28=asin(norm(pv28)/(norm(Inertialframe)*norm(S28)));
   end
   % Angle to reverse rotation
   antifi28=-fi28;
   antip028=cos(antifi28/2);
   % Calculate quaternion scalar part
   p028=cos(fi28/2);
   % Calculate quaternion vector par
   if norm(pv28)<0.0000001
       p28=transpose((pv28)*0);
       antip28=p28;
   else
       p28=transpose((pv28/norm(pv28))*sin(fi28/2));
       antip28=transpose((pv28/norm(pv28))*sin(antifi28/2));
   end

%.............................ECI to BRF.............................
   brf14=quat_rotation(p014,p14,eci14);    %SGP4 WMM    [nT]
   brf24=quat_rotation(p024,p24,eci24);    %SGP4 IGRF   [nT]
   brf18=quat_rotation(p018,p18,eci18);    %SGP8 WMM    [nT]
   brf28=quat_rotation(p028,p28,eci28);    %SGP8 IGRF   [nT]

% Adapt units form [nT] to [T]
   brf14T=brf14/(10^9);                    %SGP4 WMM    [T]
   brf24T=brf24/(10^9);                    %SGP4 IGRF   [T]
   brf18T=brf18/(10^9);                    %SGP8 WMM    [T]
   brf28T=brf28/(10^9);                    %SGP8 IGRF   [T]
```

```
%------------------Calculate mu values for the iteration-----------------
%.........................Hysteresis calculation.........................

   if type==1      %If it is hysteresis

% SGP4 WMM
       EKr14=0.5*rotspeed14*I*transpose(rotspeed14);   % Rotational kinetic
energy [J]

       DeltaEK14=Chyst*norm(brf14T)*norm(rad2deg(rotspeed14)); % Kinetic
energy variation  [J]

       EKr14=EKr14-DeltaEK14;                % Update kinetic energy [J]

       if EKr14<0                            % Verify if all has dissipated
           rotspeed14=rotspeed14*0;          % Finish the calculations
           stabilization(1)=1;               % Stabilisation to be check
       else
           Inertia=dot(S14*I,S14);           % Moment of inertia at the
axis of rotation [kg*m^2]
           rotspeed14=(S14)*sqrt(2*EKr14/Inertia); % Update rotation speed
       end

% SGP4 IGRF
       EKr24=0.5*rotspeed24*I*transpose(rotspeed24);   % Rotational kinetic
energy [J]
       DeltaEK24=Chyst*norm(brf24T)*norm(rad2deg(rotspeed24)); % Kinetic
energy variation  [J]
       EKr24=EKr24-DeltaEK24;                % Update kinetic energy [J]

       if EKr24<0                            % Verify if all has dissipated
           rotspeed24=rotspeed24*0;          % Finish the calculations
           stabilization(2)=1;               % Stabilisation to be stored
       else
           Inertia=dot(S24*I,S24);           % Moment of inertia at the
axis of rotation [kg*m^2]
           rotspeed24=(S24)*sqrt(2*EKr24/Inertia); % Update rotation speed
       end

% SGP8 WMM
       EKr18=0.5*rotspeed18*I*transpose(rotspeed18);   % Rotational kinetic
energy [J]

       DeltaEK18=Chyst*norm(brf18T)*norm(rad2deg(rotspeed18)); % Kinetic
energy variation  [J]

       EKr18=EKr18-DeltaEK18;                % Update kinetic energy [J]

       if EKr18<0                             % Verify if all has
dissipated
           rotspeed18=rotspeed18*0;          % Finish the calculations
           stabilization(3)=1;               % Stabilisation to be stored
       else
           Inertia=dot(S18*I,S18);           % Moment of inertia at the
axis of rotation [kg*m^2]
           rotspeed18=(S18)*sqrt(2*EKr18/Inertia);% Update rotation speed
       end
```

```
% SGP8 IGRF
       EKr28=0.5*rotspeed28*I*transpose(rotspeed28);   % Rotational kinetic
energy [J]

       DeltaEK28=Chyst*norm(brf28T)*norm(rad2deg(rotspeed28)); % Kinetic
energy variation  [J]

       EKr28=EKr28-DeltaEK28;                % Update kinetic energy [J]

       if EKr28<0                            % Verify if all has dissipated
           rotspeed28=rotspeed28*0;          % Finish the calculations
           stabilization(4)=1;               % Stabilisation to be stored
       else
           Inertia=dot(S28*I,S28);           % Moment of inertia at the
axis of rotation [kg*m^2]
           rotspeed28=(S28)*sqrt(2*EKr28/Inertia); % Update rotation speed
       end

% Save stabilisation dates
       if stabilization(1)==1
           stabdate(1)=t/1440;
           stabilization(1)=stabilization(1)+1;
       end

       if stabilization(2)==1
           stabdate(2)=t/1440;
           stabilization(2)=stabilization(2)+1;
       end

       if stabilization(3)==1
           stabdate(3)=t/1440;
           stabilization(3)=stabilization(3)+1;
       end

       if stabilization(4)==1
           stabdate(4)=t/1440;
           stabilization(4)=stabilization(4)+1;
       end

       if stabilization(1)>1 && stabilization(2)>1 && stabilization(3)>1 &&
stabilization(4)>1
           t=tsince;
       end
%.......................Magnetorquer calculation.......................
   elseif type==2              % If it is magnetorquer

% SGP4 WMM
       acceleration14=(-rotspeed14)/step_sec;      % [rad/s^2]
       Torque14=I*transpose(acceleration14);       % [N*m]
       muhat14=cross(brf14T,Torque14)/norm(cross(brf14T,Torque14));
       k14=((dot(muhat14,brf14T)*brf14T)/dot(brf14T,brf14T))-muhat14;
       mu14=(norm(cross(Torque14,brf14T))/(norm(k14)*dot(brf14T,brf14T)));
       mu14=mu14*muhat14;                          % [A*m^2]
       % Verify mu is not over the max value
       if (abs(mu14(1))+abs(mu14(2))+abs(mu14(3)))>mumax
           muk=mumax/(abs(mu14(1))+abs(mu14(2))+abs(mu14(3)));
           mu14(1)=mu14(1)*muk;
           mu14(2)=mu14(2)*muk;
           mu14(3)=mu14(3)*muk;
       end
```

```
% SGP4 IGRF
       acceleration24=(-rotspeed24)/step_sec;      % [rad/s^2]
       Torque24=I*transpose(acceleration24);       % [N*m]
       muhat24=cross(brf24T,Torque24)/norm(cross(brf24T,Torque24));
       k24=((dot(muhat24,brf24T)*brf24T)/dot(brf24T,brf24T))-muhat24;
       mu24=(norm(cross(Torque24,brf24T))/(norm(k24)*dot(brf24T,brf24T)));
       mu24=mu24*muhat24;                          % [A*m^2]
       % Verify mu is not over the max value
       if (abs(mu24(1))+abs(mu24(2))+abs(mu24(3)))>mumax
           muk=mumax/(abs(mu24(1))+abs(mu24(2))+abs(mu24(3)));
           mu24(1)=mu24(1)*muk;
           mu24(2)=mu24(2)*muk;
           mu24(3)=mu24(3)*muk;
       end

% SGP8 WMM
       acceleration18=(-rotspeed18)/step_sec;      % [rad/s^2]
       Torque18=I*transpose(acceleration18);       % [N*m]
       muhat18=cross(brf18T,Torque18)/norm(cross(brf18T,Torque18));
       k18=((dot(muhat18,brf18T)*brf18T)/dot(brf18T,brf18T))-muhat18;
       mu18=(norm(cross(Torque18,brf18T))/(norm(k18)*dot(brf18T,brf18T)));
       mu18=mu18*muhat18;                          % [A*m^2]
       % Verify mu is not over the max value
       if (abs(mu18(1))+abs(mu18(2))+abs(mu18(3)))>mumax
           muk=mumax/(abs(mu18(1))+abs(mu18(2))+abs(mu18(3)));
           mu18(1)=mu18(1)*muk;
           mu18(2)=mu18(2)*muk;
           mu18(3)=mu18(3)*muk;
       end

% SGP8 IGRF
       acceleration28=(-rotspeed28)/step_sec;      % [rad/s^2]
       Torque28=I*transpose(acceleration28);       % [N*m]
       muhat28=cross(brf28T,Torque28)/norm(cross(brf28T,Torque28));
       k28=((dot(muhat28,brf28T)*brf28T)/dot(brf28T,brf28T))-muhat28;
       mu28=(norm(cross(Torque28,brf28T))/(norm(k28)*dot(brf28T,brf28T)));
       mu28=mu28*muhat28;                          % [A*m^2]
       if (abs(mu28(1))+abs(mu28(2))+abs(mu28(3)))>mumax
           muk=mumax/(abs(mu28(1))+abs(mu28(2))+abs(mu28(3)));
           mu28(1)=mu28(1)*muk;
           mu28(2)=mu28(2)*muk;
           mu28(3)=mu28(3)*muk;
       end



%--------------------Calculate rotation of the main axis-------------------

% Calculate torques
       T14=cross(mu14,brf14T);                        %[N*m] SGP4 WMM
       T24=cross(mu24,brf24T);                        %[N*m] SGP4 IGRF
       T18=cross(mu18,brf18T);                        %[N*m] SGP8 WMM
       T28=cross(mu28,brf28T);                        %[N*m] SGP8 IGRF

% Calculate acceleration vectors
       acceleration14=transpose(linsolve(I,T14));     %[rad/s^2] SGP4 WMM
       acceleration24=transpose(linsolve(I,T24));     %[rad/s^2] SGP4 IGRF
       acceleration18=transpose(linsolve(I,T18));     %[rad/s^2] SGP8 WMM
       acceleration28=transpose(linsolve(I,T28));     %[rad/s^2] SGP8 IGRF
```

```
% Calculate angle turned
       % SGP4 WMM [rad]
       theta14=rotspeed14*step_sec+(1/2)*acceleration14*step_sec*step_sec;
       % SGP4 IGRF [rad]
       theta24=rotspeed24*step_sec+(1/2)*acceleration24*step_sec*step_sec;
       % SGP8 WMM [rad]
       theta18=rotspeed18*step_sec+(1/2)*acceleration18*step_sec*step_sec;
       % SGP8 IGRF [rad]
       theta28=rotspeed28*step_sec+(1/2)*acceleration28*step_sec*step_sec;

% Calculate the rotation axis for the iteration   % [unit vector]
       rotaxis14=rotspeed14/norm(rotspeed18);  % SGP4 WMM
       rotaxis24=rotspeed24/norm(rotspeed24);  % SGP4 IGRF
       rotaxis18=rotspeed18/norm(rotspeed18);  % SGP8 WMM
       rotaxis28=rotspeed28/norm(rotspeed28);  % SGP8 IGRF

% Calculate the step quaternion
% SGP4 WMM
       q014=cos(norm(theta14)/2);              % Quaternion scalar part
       q14=rotaxis14*(sin(norm(theta14)/2));   % Quaternion vector part

% SGP4 IGRF
       q024=cos(norm(theta24)/2);              % Quaternion scalar part
       q24=rotaxis24*(sin(norm(theta24)/2));   % Quaternion vector part

% SGP8 WMM
       q018=cos(norm(theta18)/2);              % Quaternion scalar part
       q18=rotaxis18*(sin(norm(theta18)/2));   % Quaternion vector part

   % SGP8 IGRF
       q028=cos(norm(theta28)/2);              % Quaternion scalar part
       q28=rotaxis28*(sin(norm(theta28)/2));   % Quaternion vector part

% Transform satellites main axis to ECI
   % SGP4 WMM
       Sx14=quat_rotation(q014,q14,Sx14);      % Satellite X axis in ECI
       Sy14=quat_rotation(q014,q14,Sy14);      % Satellite Y axis in ECI
       Sz14=quat_rotation(q014,q14,Sz14);      % Satellite Z axis in ECI

   % SGP4 IGRF
       Sx24=quat_rotation(q024,q24,Sx24);      % Satellite X axis in ECI
       Sy24=quat_rotation(q024,q24,Sy24);      % Satellite Y axis in ECI
       Sz24=quat_rotation(q024,q24,Sz24);      % Satellite Z axis in ECI

   % SGP8 WMM
       Sx18=quat_rotation(q018,q18,Sx18);      % Satellite X axis in ECI
       Sy18=quat_rotation(q018,q18,Sy18);      % Satellite Y axis in ECI
       Sz18=quat_rotation(q018,q18,Sz18);      % Satellite Z axis in ECI

   % SGP8 IGRF
       Sx28=quat_rotation(q028,q28,Sx28);      % Satellite X axis in ECI
       Sy28=quat_rotation(q028,q28,Sy28);      % Satellite Y axis in ECI
       Sz28=quat_rotation(q028,q28,Sz28);      % Satellite Z axis in ECI

% Calculate new speed of rotation
       rotspeed14=rotspeed14+acceleration14*step_sec;  % [rad/s] SGP4 WMM
       rotspeed24=rotspeed24+acceleration24*step_sec;  % [rad/s] SGP4 IGRF
       rotspeed18=rotspeed18+acceleration18*step_sec;  % [rad/s] SGP8 WMM
       rotspeed28=rotspeed28+acceleration28*step_sec;  % [rad/s] SGP8 IGRF
   else
```

```
        fprintf('Wrong type of system, type must be between 1 and 3');
        t=tsince;
    end

%-------------------------------Save data---------------------------------
% Rotation rates

% SGP4 WMM
    omega14X(iteration)=rad2deg(rotspeed14(1));     % [deg/s]
    omega14Y(iteration)=rad2deg(rotspeed14(2));     % [deg/s]
    omega14Z(iteration)=rad2deg(rotspeed14(3));     % [deg/s]
    omega14M(iteration)=rad2deg(norm(rotspeed14));  % [deg/s]

% SGP4 IGRF
    omega24X(iteration)=rad2deg(rotspeed24(1));     % [deg/s]
    omega24Y(iteration)=rad2deg(rotspeed24(2));     % [deg/s]
    omega24Z(iteration)=rad2deg(rotspeed24(3));     % [deg/s]
    omega24M(iteration)=rad2deg(norm(rotspeed24));  % [deg/s]

% SGP8 WMM
    omega18X(iteration)=rad2deg(rotspeed18(1));     % [deg/s]
    omega18Y(iteration)=rad2deg(rotspeed18(2));     % [deg/s]
    omega18Z(iteration)=rad2deg(rotspeed18(3));     % [deg/s]
    omega18M(iteration)=rad2deg(norm(rotspeed18));  % [deg/s]

% SGP8 IGRF
    omega28X(iteration)=rad2deg(rotspeed28(1));     % [deg/s]
    omega28Y(iteration)=rad2deg(rotspeed28(2));     % [deg/s]
    omega28Z(iteration)=rad2deg(rotspeed28(3));     % [deg/s]
    omega28M(iteration)=rad2deg(norm(rotspeed28));  % [deg/s]

% Store power consumption for magnetorquers [mW]

    if type==2
        powerc14(iteration)=((mu14(1)^2)+(mu14(2)^2)+(mu14(3))^2)/(powrconst
^2);

        powerc24(iteration)=((mu24(1)^2)+(mu24(2)^2)+(mu24(3))^2)/(powrconst
^2);

        powerc18(iteration)=((mu18(1)^2)+(mu18(2)^2)+(mu18(3))^2)/(powrconst
^2);

        powerc28(iteration)=((mu28(1)^2)+(mu28(2)^2)+(mu28(3)^2))/(powrconst
^2);
    end

% Store time value
    timed(iteration)=t/1440;                        %[days]
    timeh(iteration)=t/60;                          %[hours]
    timem(iteration)=t;                             %[min]

%-----------------------------Update counter------------------------------

    iteration                       % Print the iteration number
    iteration=iteration+1;
    t=t+step;
end
```

```
%-------------------------------------------------------------------------
%-----------------------------Generate plots------------------------------
%-------------------------------------------------------------------------

if type==1 % Plot if valid hysteresis calculation

   figure
   plot(timeh,omega28M,timeh,omega18M,timeh,omega24M,timeh,omega14M,'LineWi
dth',1.1)
   title('Hysteresis 500 km')
   xlabel('Time elapsed [hours]')
   ylabel('Speed of rotation [deg/s]')
   legend('SGP8 IGRF','SGP8 WMM','SGP4 IGRF','SGP4 WMM')
   set(gca,'linewidth',1.5)

% Print the hysteresis stabilization time
   if  stabilization(1)==2
       fprintf('Satellite hysteresis stabilised after %.4f days. SGP4
WMM\n',stabdate(1));
   else
       fprintf('Satellite hysteresis does not stabilise after %.4f days.
SGP4 WMM\n',tsince);
   end

   if  stabilization(2)==2
       fprintf('Satellite hysteresis stabilised after %.4f days. SGP4
IGRF\n',stabdate(2));
       else
       fprintf('Satellite hysteresis does not stabilise after %.4f days.
SGP4 IGRF\n',tsince);
   end

   if  stabilization(3)==2
       fprintf('Satellite hysteresis stabilised after %.4f days. SGP8
WMM\n',stabdate(3));
       else
       fprintf('Satellite hysteresis does not stabilise after %.4f days.
SGP8 WMM\n',tsince);
   end

   if  stabilization(4)==2
       fprintf('Satellite hysteresis stabilised after %.4f days. SGP8
IGRF\n',stabdate(4));
       else
       fprintf('Satellite hysteresis does not stabilise after %.4f days.
SGP4 IGRF\n',tsince);
   end

elseif type==2 % Plot if valid magnetorquer calculation

   figure
   plot(timem,omega28M,timem,omega18M,timem,omega24M,timem,omega14M,'LineWi
dth',1.1)
   title('Magnetorquers 500 km')
   xlabel('Time elapsed [min]')
   ylabel('Speed of rotation [deg/s]')
   legend('SGP8 IGRF','SGP8 WMM','SGP4 IGRF','SGP4 WMM')
   set(gca,'linewidth',1.5)
```

```
   figure
   plot(timem,powerc24,timem,powerc14,timem,powerc28,timem,powerc18,'LineWi
dth',1.1)
   title('Power consumed 500 km')
   xlabel('Time elapsed [min]')
   ylabel('Power consumed [mW]')
   legend('SGP4 IGRF','SGP4 WMM','SGP8 IGRF','SGP8 WMM')
   set(gca,'linewidth',1.5)

else        %Error in the type selection

   fprintf('Wrong type of system, type must be between 1 and 3');
end
```

Propagator verification code

```
%-------------------------------------------------------------------------
%                     Propagator verification subcode
%
%-------------------------------------------------------------------------

clc
clear
format long g

%-------------------------------------------------------------------------
%----------------------------------INPUTS---------------------------------
%-------------------------------------------------------------------------

% TLE files names
fname = 'SATELIOT_1.txt';

% Propagation parameters
propdays=20;                % Propagation time [days]
step_sec = 300;             % Step [seconds]

%-------------------------------------------------------------------------
%-------------------------Get initial parameters--------------------------
%-------------------------------------------------------------------------

% Obtain orbit data form TLE
[satdata]= Orbitalp (fname);

% Time parameters for the propagation of satellite's state vector
% forward (+) or backward (-) [minutes]

tsince = propdays*1440;     % Propagation time [minutes]
step=step_sec/60;           % Step [minutes]
t=step;                     % First step
len=fix(tsince/step)-1;     % Number of iterations

%----------------------Generate the storage arrays----------------------
% Difference in position storage
Error_X(len)=0;             % Storage for X value
Error_Y(len)=0;             % Storage for Y value
Error_Z(len)=0;             % Storage for Z value
Error_Norm(len)=0;          % Storage for module

% Store time of the iteration
time(len)=0;

% Iteration counter initialization
iteration=1;

%-------------------------Get the beginning year--------------------------

if (satdata.year< 57)
   year0 = satdata.year + 2000;
else
   year0 = satdata.year + 1900;
end
```

```
%-------------------------------------------------------------------------
%-----------------------------------Loop----------------------------------
%-------------------------------------------------------------------------

while t<tsince

%------------------------------Time update--------------------------------
   % Calculate date for iteration
   if satdata.doy+(t/1440) < 365
       doy = (satdata.doy+(t/1440));
       year=year0;
   else
       doy = (satdata.doy+(t/1440)-365);
       year=year0+1;
   end

   % Calculate date parameters
   [mon,day,hr,minute,sec] = days2mdh(year,doy);

%-------------------------Propagate the orbit-----------------------------
   [rteme4, vteme4] = sgp4(t, satdata);        % with SGP4 [km, km/s]
   [rteme8, vteme8] = sgp8(t, satdata);        % with SGP8 [km, km/s]

%-----------------------------Save test data------------------------------
% Store difference percentage between propagators
   % X component
   Error_X(iteration)=abs((rteme8(1)-rteme4(1))/rteme8(1))*100;
   % Y componen
   Error_Y(iteration)=abs((rteme8(2)-rteme4(2))/rteme8(2))*100;
   % Z componen
   Error_Z(iteration)=abs((rteme8(3)-rteme4(3))/rteme8(3))*100;
   % Modulus
   Error_Norm(iteration)=abs((norm(rteme8)-norm(rteme4))/norm(rteme8))*100;

% Store time value
   time(iteration)=t/1440;                     % [days]

%----------------------------Update counter-------------------------------
   iteration       % Print the iteration number
   iteration=iteration+1;
   t=t+step;
end

%-------------------------------------------------------------------------
%----------------------------Generate plots-------------------------------
%-------------------------------------------------------------------------
figure
plot(time,Error_X,'r',time,Error_Y,'b',time,Error_Z,'g',"LineWidth",1)
title('Position vector error between SGP4 and SGP8')
xlabel('Time elapsed [days]')
ylabel('% of error')
legend('Error X','ErrorY','Error Z')

figure
plot(time,Error_Norm)
title('Position vector error between SGP4 and SGP8')
xlabel('Time elapsed [days]')
ylabel('% of error')
```

Magnetic field verification code

```
%-------------------------------------------------------------------------
%                   Magnetic field verification subcode
%
%-------------------------------------------------------------------------

clc
clear
format long g

%-------------------------------------------------------------------------
%----------------------------------INPUTS---------------------------------
%-------------------------------------------------------------------------

% TLE files names
fname = 'SATELIOT_1.txt';

% Propagation parameters
propdays=20;                % Propagation time [days]
step_sec = 300;             % Step [seconds]

%-------------------------------------------------------------------------
%-------------------------Get initial parameters--------------------------
%-------------------------------------------------------------------------

% Obtain orbit data form TLE
[satdata]= Orbitalp (fname);

% Time parameters for the propagation of satellite's state vector
% forward (+) or backward (-) [minutes]

tsince = propdays*1440;     % Propagation time [minutes]
step=step_sec/60;           % Step [minutes]
t=step;                     % First step
len=fix(tsince/step)-1;     % Number of iterations

%-----------------------Generate the storage arrays-----------------------
% Diference in magnetic field storage
Error_X(len)=0;             % Storage for X value
Error_Y(len)=0;             % Storage for Y value
Error_Z(len)=0;             % Storage for Z value
Error_M(len)=0;             % Storage for module

% Store time of the iteration
time(len)=0;

% Iteration counter initialization
iteration=1;

%-------------------------Get the beginning year--------------------------

if (satdata.year< 57)
   year0 = satdata.year + 2000;
else
   year0 = satdata.year + 1900;
end
```

```
%-------------------------------------------------------------------------
%-----------------------------------Loop----------------------------------
%-------------------------------------------------------------------------

while t<tsince

%------------------------------Time update--------------------------------
   % Calculate date for iteration
   if satdata.doy+(t/1440) < 365
       doy = (satdata.doy+(t/1440));
       year=year0;
   else
       doy = (satdata.doy+(t/1440)-365);
       year=year0+1;
   end

   % Calculate date parameters
   [mon,day,hr,minute,sec] = days2mdh(year,doy);
   % Date as decimal year
   dy = decyear(year,mon,day,hr,minute,sec);
   % Generate UTC vector
   utc=[year,mon,day,hr,minute,sec];

%--------------------------Propagate the orbit----------------------------
   [rteme, vteme] = sgp8(t, satdata);  % [km, km/s]
% Convert position vector units from [km] to [m]
   rteme=rteme*1000;                    % [m]
% Calculate LLA coordinates
   lla=eci2lla([rteme(1),rteme(2),rteme(3)],utc);
   lat=lla(1);    lon=lla(2);    h=lla(3);

%------------------------Calculate magnetic fields------------------------
% SGP8  [nT]
   [XYZWMM, HWMM, DWMM, IWMM, FWMM] = wrldmagm(h,lat,lon,dy,'2020'); % WMM
   [XYZIGRF,HIGRF,DIGRF,IIGRF,FIGRF] = igrfmagm(h,lat,lon,dy,13);    % IGRF
   XYZIGRF=transpose(XYZIGRF);       %Adapt magnetic field vector format

%----------------------------Save test data-------------------------------
% Store difference percentage between magnetic fields
   % X component
   Error_X(iteration)=abs(((XYZWMM(1)-XYZIGRF(1))/XYZWMM(1))*100);

   % Y component
   Error_Y(iteration)=abs(((XYZWMM(2)-XYZIGRF(2))/XYZWMM(2))*100);

   % Z component
   Error_Z(iteration)=abs(((XYZWMM(3)-XYZIGRF(3))/XYZWMM(3))*100);

   % Modulus
   Error_M(iteration)=abs(((FWMM-FIGRF)/FWMM)*100);
% Store time value
   time(iteration,1)=t/1440;           % [days]

%----------------------------Update counter-------------------------------
   iteration               % Print the iteration number
   iteration=iteration+1;
   t=t+step;
end
```

```
%--------------------------------------------------------------------------
%------------------------------Generate plots------------------------------
%--------------------------------------------------------------------------

% Components
figure
plot(time,Error_X,time,Error_Y,time,Error_Z,)
title('Error between IGRF and WMM')
xlabel('Time elapsed')
ylabel('% of error')
legend('Error X','ErrorY','Error Z')

% Modulus
figure
plot(time,Error_M)
title('Total Error between IGRF and WMM')
xlabel('Time elapsed')
ylabel('% of error')
```

Magnetic field vector rotation verification code

```
%-------------------------------------------------------------------------
%                       Rotation verification subcode
%
%-------------------------------------------------------------------------

clc
clear
format long g

%-------------------------------------------------------------------------
%---------------------------------INPUTS----------------------------------
%-------------------------------------------------------------------------

% TLE files names
fname = 'SATELIOT_1.txt';

% Tensor of Inertia [g*mm^2]
I=[[37971.2,-17.4,-15.7];
   [-17.4,37596.6,238.8];
   [-15.7,238.8,37727.4]];

% Initial rotation speed
rotspeed=[1,1,1]*deg2rad(20);       % [rad/s]

% Propagation parameters
propdays= 0.01;                     % propagation time [days]
step_sec = 1;                       % step [seconds]

% Dipolar moment
mu=[0,0.2,0];                       % [A*m^2]

% Satellite main axis at ECI frame at t=0
Sx=[1,0,0];                         % X axis [unit]
Sy=[0,1,0];                         % Y axis [unit]
Sz=[0,0,1];                         % Z axis [unit]

%-------------------------------------------------------------------------
%-------------------------Get initial parameters--------------------------
%-------------------------------------------------------------------------

% Reference vector for the ECI frame coordinate system
Inertialframe=[1,1,1]/(sqrt(3));    % [unit]

% Obtain orbit data form TLE
[satdata]= Orbitalp (fname);

% Change Tensor of Inertia in units from [g*mm^2] to [kg*m^2]
I=I/(10^9);                         % [kg*m^2]

% Time parameters for the propagation of satellite's state vector
% forward (+) or backward (-) [minutes]

tsince = propdays*1440;             % Propagation time [minutes]
step=step_sec/60;                   % Step [minutes]
t=0;                                % First step
len=fix(tsince/step)-1;             % Number of iterations
```

```
%-----------------------Generate the storage arrays-----------------------
% Diference in rotation of the magnetic field storage
test1(len)=0;                       % Storage for error from NED to ECEF
test2(len)=0;                       % Storage for error from ECEF to ECI
test3(len)=0;                       % Storage for error from ECI to BRF

% Store time of the iteration
time(len)=0;

% Iteration counter initialization
iteration=1;

%-------------------------Get the beginning year-------------------------
if (satdata.year< 57)
   year0 = satdata.year + 2000;
else
   year0 = satdata.year + 1900;
end

%-------------------------------------------------------------------------
%-----------------------------------Loop----------------------------------
%-------------------------------------------------------------------------
while t<tsince

%-------------------------------Time update-------------------------------
   % Calculate date for iteration
   if satdata.doy+(t/1440) < 365
       doy = (satdata.doy+(t/1440));
       year=year0;
   else
       doy = (satdata.doy+(t/1440)-365);
       year=year0+1;
   end

   % Calculate date parameters
   [mon,day,hr,minute,sec] = days2mdh(year,doy);

   % Date as decimal year
   dy = decyear(year,mon,day,hr,minute,sec);

   % Generate UTC vector
   utc=[year,mon,day,hr,minute,sec];

%-------------------------Propagate the orbit-------------------------
   [rteme, vteme] = sgp8(t, satdata);  % [km, km/s]

% Convert position vector units from [km] to [m]
   rteme=rteme*1000;                    % [m]

% Calculate LLA coordinates
   lla=eci2lla([rteme(1),rteme(2),rteme(3)],utc);
   lat=lla(1);    lon=lla(2);    h=lla(3);

%-----------------------Calculate magnetic fields-----------------------
% SGP8  [nT]
   [XYZ,H2,D2,I2,F2] = igrfmagm(h,lat,lon,dy,13);         %IGRF
   XYZ=transpose(XYZ);     % Adapt magnetic field vector format
```

```
%-------------------Transform magnetic fields into BRF-------------------
%.............................NED to ECEF.............................
   [ECEF(1),ECEF(2),ECEF(3)]=ned2ecefv(XYZ(1),XYZ(2),XYZ(3),lat,lon);%[nT]

%.............................ECEF to ECI.............................
   ECI=ecef2eci(utc,ECEF);         % [nT]

%.................Quaternion for ECI to BRF rotation..................
   S=Sx+Sy+Sz;                     % Generate satellite orientation vector
   S=S/norm(S);                    % Normalise orientation vector
   pv=cross(Inertialframe,S);      % Calculate rotation axis
   % Calculate rotation angle
   if S(1)+S(2)+S(3)<0
       fi=deg2rad(180)-(asin(norm(pv))/(norm(Inertialframe)*norm(S)));
   else
       fi=asin(norm(pv)/(norm(Inertialframe)*norm(S)));
   end
   % Angle to reverse rotation
   antifi=-fi;

   % Calculate quaternion scalar part
   p0=cos(fi/2);
   antip0=cos(antifi/2);
   % Calculate quaternion vector part
   if norm(pv)<0.0000001
       p=transpose((pv)*0);
       antip=p;
   else
       p=transpose((pv/norm(pv))*sin(fi/2));
       antip=transpose((pv/norm(pv))*sin(antifi/2));
   end

%.............................ECI to BRF.............................
   BRF=quat_rotation(p0,p,ECI);                % [nT]
   antiBRF=quat_rotation(antip0,antip,BRF);

% Adapt units form [nT] to [T]
   BRF_tesla=BRF/(10^9);                       % [T]

%-------------------Calculate rotation of the main axis------------------
% Calculate torques
   T=transpose(cross(mu,BRF_tesla));           % [N*m]

% Calculate acceleration vectors
   acceleration=transpose(linsolve(I,T));      % [rad/s^2]

% Rotate acceleration from BRF back to ECI
   acceleration=quat_rotation(antip0,antip,acceleration);

% Calculate angle turned
   theta=rotspeed*step_sec+(1/2)*acceleration*step_sec*step_sec;   % [rad]

% Rotation axis for the iteration
   rotaxis=rotspeed/norm(rotspeed);            % [unitary vector]

% Calculate the step quaternion [To be reviewed]
   q0=cos(norm(theta)/2);                      % Quaternion scalar part
   q=rotaxis*(sin(norm(theta)/2));             % Quaternion vector part
```

```
% Transform satellites main axis to ECI
   Sx=quat_rotation(q0,q,Sx);                  % Satellite X axis in ECI
   Sy=quat_rotation(q0,q,Sy);                  % Satellite Y axis in ECI
   Sz=quat_rotation(q0,q,Sz);                  % Satellite Z axis in ECI

% Calculate new speed of rotation
   rotspeed=rotspeed+acceleration*step_sec;   % [rad/s]

%------------------------------Save test data-------------------------------
% Store rotation error in percentage
   % NED to ECEF
   test1(iteration)=abs((norm(XYZ)-norm(ECEF))/norm(XYZ))*100;
   % ECEF to ECI
   test2(iteration)=abs((norm(ECEF)-norm(ECI))/norm(ECEF))*100;
   % ECI to BRF
   test3(iteration)=abs((norm(ECI)-norm(BRF))/norm(ECI))*100;

% Store time value
   time(iteration)=t/1440;                     % [days]

%------------------------------Update counter-------------------------------

   iteration       % Print the iteration number
   iteration=iteration+1;
   t=t+step;
end

%--------------------------------------------------------------------------
%------------------------------Generate plots------------------------------
%--------------------------------------------------------------------------

% NED to ECEF
figure
plot(time,test1)
title('Error NED to ECEF')
xlabel('Time elapsed [days]')
ylabel('% of error')

% ECEF to ECI
figure
plot(time,test2)
title('Error ECEF to ECI')
xlabel('Time elapsed [days]')
ylabel('% of error')

% ECI to BRF
figure
plot(time,test3)
title('Error ECI to BRF')
xlabel('Time elapsed [days]')
ylabel('% of error')
```

TLE orbital parameters code

```
%-------------------------------------------------------------------
%-------------- Obtain orbital parameters form TLE -----------------
%-------------------------------------------------------------------
function [satdata] = Orbitalp(fname)
ge = 398600.8; % Earth gravitational constant [km^3/s^2]
TWOPI = 2*pi;
MINUTES_PER_DAY = 1440.;
MINUTES_PER_DAY_SQUARED = (MINUTES_PER_DAY * MINUTES_PER_DAY);
MINUTES_PER_DAY_CUBED = (MINUTES_PER_DAY * MINUTES_PER_DAY_SQUARED);
% Open the TLE file and read TLE elements
fid = fopen(fname, 'r');
% read first line
tline = fgetl(fid);
Cnum = tline(3:7);                                  % Catalogue Number (NORAD)
SC   = tline(8);                                 % Security Classification
ID   = tline(10:17);                             % Identification Number
year = str2num(tline(19:20));                     % Year
doy  = str2num(tline(21:32));                     % Day of year
epoch = str2num(tline(19:32));                    % Epoch
TD1   = str2num(tline(34:43));                    % first time derivative
TD2   = str2num(tline(45:50));                    % 2nd Time Derivative
ExTD2 = str2num(tline(51:52));                    % Exponent of 2nd Time
Derivative
BStar = str2num(tline(54:59));                    % Bstar/drag Term
ExBStar = str2num(tline(60:61));                  % Exponent of Bstar/drag Term
BStar = BStar*1e-5*10^ExBStar;
Etype = tline(63);                                % Ephemeris Type
Enum  = str2num(tline(65:end));                   % Element Number
% read second line
tline = fgetl(fid);
i = str2num(tline(9:16));                         % Orbit Inclination (degrees)
raan = str2num(tline(18:25));                     % Right Ascension of Ascending
Node (degrees)
e = str2num(strcat('0.',tline(27:33)));           % Eccentricity
omega = str2num(tline(35:42));                    % Argument of Perigee (degrees)
M = str2num(tline(44:51));                        % Mean Anomaly (degrees)
no = str2num(tline(53:63));                       % Mean Motion
a = ( ge/(no*2*pi/86400)^2 )^(1/3);               % semi major axis (km)
rNo = str2num(tline(65:end));                     % Revolution Number at Epoch
fclose(fid);
satdata.epoch = epoch;
satdata.year = year;
satdata.doy = doy;
satdata.norad_number = Cnum;
satdata.bulletin_number = ID;
satdata.classification = SC;                      % almost always 'U'
satdata.revolution_number = rNo;
satdata.ephemeris_type = Etype;
satdata.xmo = M * (pi/180);
satdata.xnodeo = raan * (pi/180);
satdata.omegao = omega * (pi/180);
satdata.xincl = i * (pi/180);
satdata.eo = e;
satdata.xno = no * TWOPI / MINUTES_PER_DAY;
satdata.semimajor = a;
satdata.xndt2o = TD1 * TWOPI / MINUTES_PER_DAY_SQUARED;
satdata.xndd6o = TD2 * 10^ExTD2 * TWOPI / MINUTES_PER_DAY_CUBED;
satdata.bstar = BStar;
```

Quaternion rotation

```
%-------------------------------------------------------------------------
%-----------------------Quaternion rotation-------------------------------
%-------------------------------------------------------------------------

function [r]=quat_rotation(p0,p,v)
r= v + (2*p0*cross(p,v)) + cross((2*p),(cross(p,v)));
```